

平安消费金融高并发业务架构 演进实践与思考

张鑫

平安银行 消费金融事业部架构师



精彩继续！ 更多一线大厂前沿技术案例

📍 北京站

QCon

全球软件开发大会

时间：2021年5月29-31日

地点：北京 · 国际会议中心

扫码查看大会
详情>>



📍 北京站

GMITC

全球大前端技术大会

时间：2021年6月25-26日

地点：北京 · 国际会议中心

扫码查看大会
详情>>



📍 深圳站

ArchSummit

全球架构师峰会

时间：2021年7月9-10日

地点：深圳 · 大中华喜来登酒店

扫码查看大会
详情>>



自我介绍



- 张鑫
- 平安银行消费金融事业部架构师
- 负责消费贷款平台架构建设、升级、性能优化等工作
- 平台支撑平安个人贷款业务年均50%+复合增长、日均贷款量10亿+
- 个人专栏：《高并发互联网消费金融领域架构设计》

<https://blog.51cto.com/cloumn/detail/89>

大纲

- 互联网消费金融架构的特性及面临的挑战
- 平安消费金融平台的服务化架构方案演进（单体–SOA–微服务）
- 平安消费金融平台微服务中间件选型及演进
- 平安消费金融平台架构演进面临的常见问题及思考

互联网消费信贷的常用模式

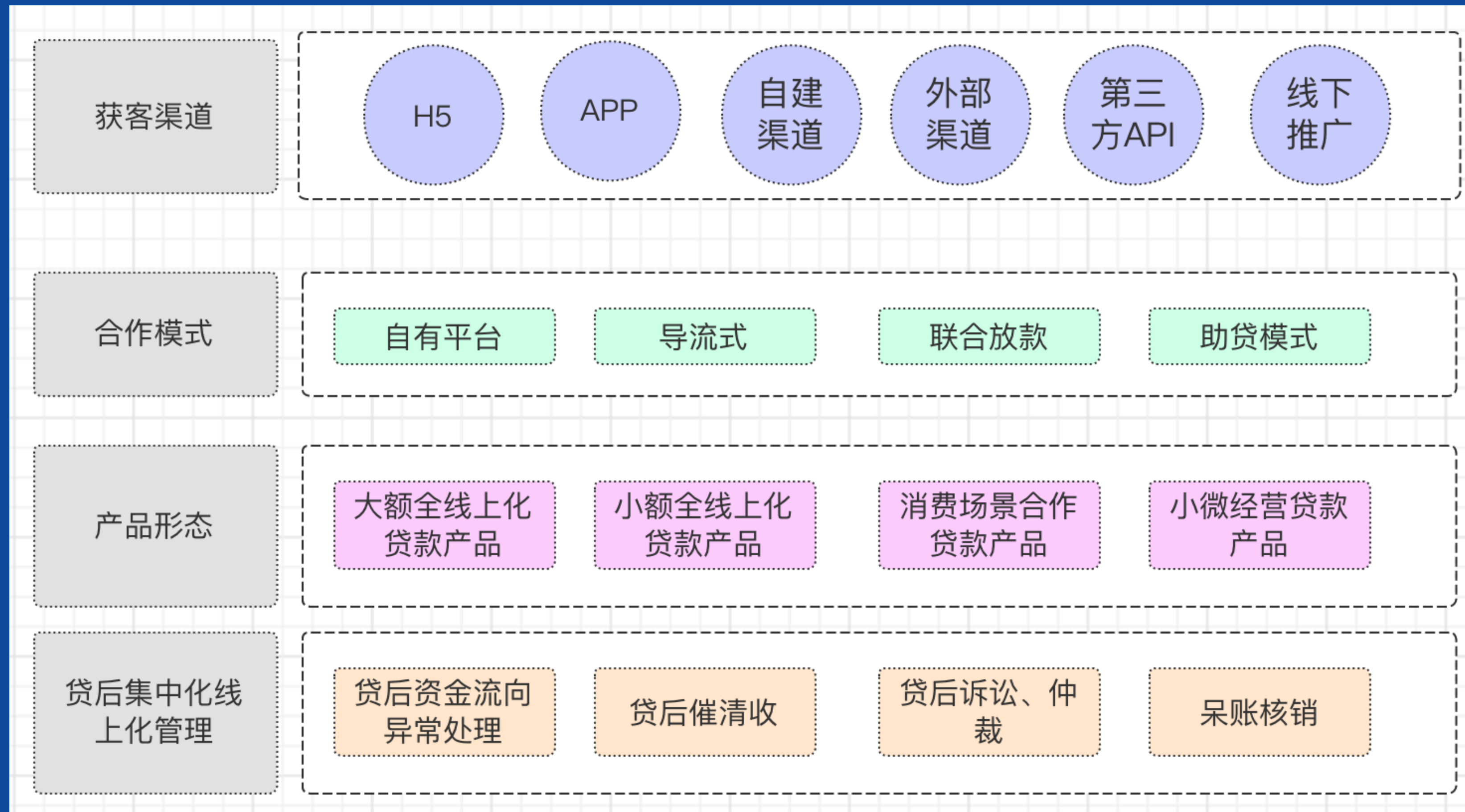
➤ 互联网消费信贷常见模式有三大类：

- 小额现金贷
- 常规信用贷
- 场景消费贷



发展趋势：强监管、P2P已消失

互联网消费金融的产品特点



互联网消费金融的产品要素

授信对象

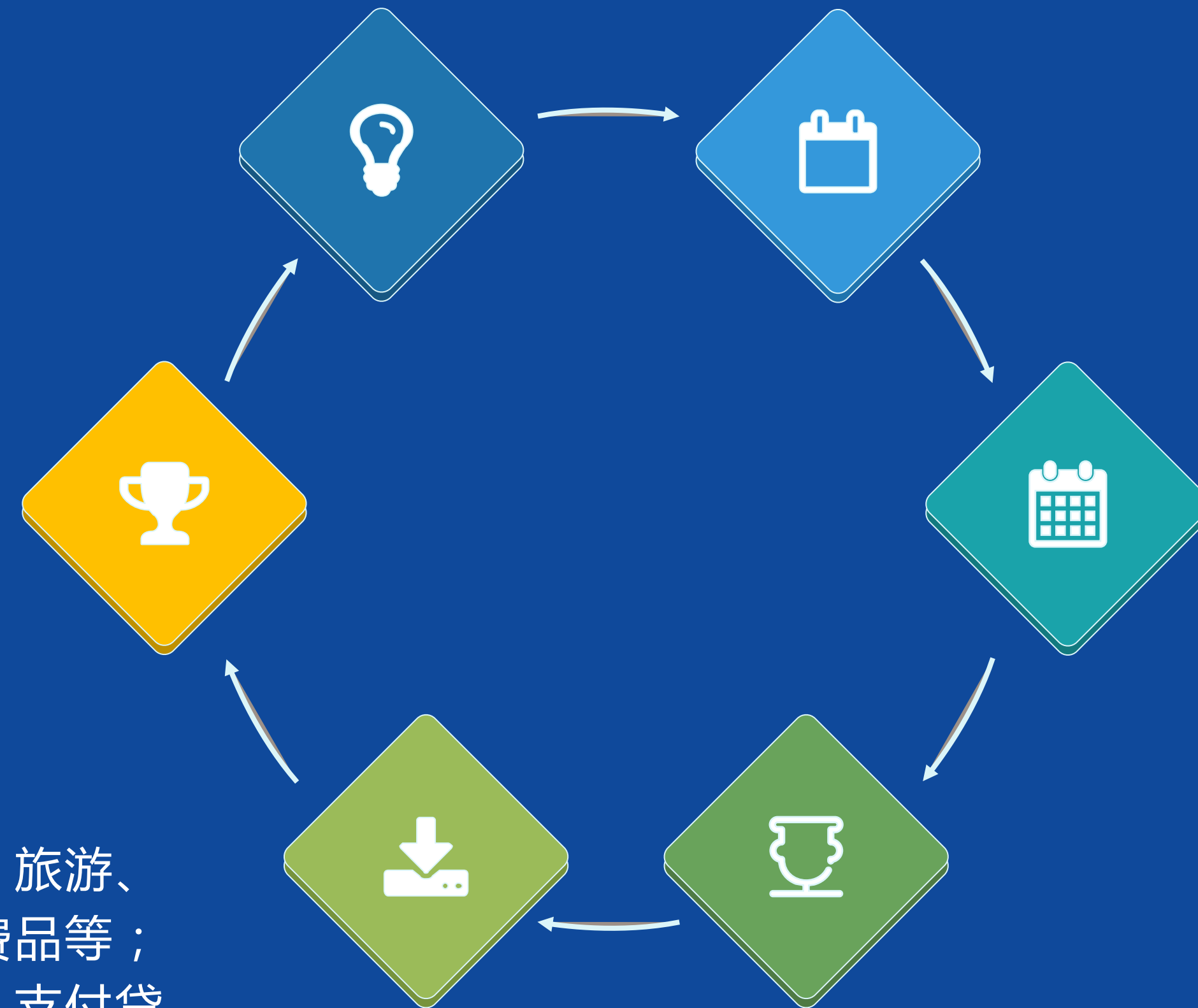
分为**个人**（自然人）贷款、**企业**（法人）贷款；

担保方式

抵押贷款、**质押**贷款
保证贷款、**信用**贷款

贷款用途

消费贷款：购买住房、装修、购车、旅游、进修、购买大额消费品等；
经营贷款：购买商用房、购置设备、支付贷款、支付雇员工资等。



金额

授信额度：对整体项目（如按揭楼盘）或单一对象授信，额度下可包含多笔贷款；**贷款金额**：1-50万不等；

利率

固定利率贷款：贷款期限内不调整利率；
浮动利率贷款：随基准利率调整，如按揭。调整方式有年调、季调、月调；

期限

短期贷款（1年以内）
中期贷款（1-5年）
长期贷款（5年以上）

银行架构体系的设计原则



稳定

高可用
低故障



监管合规

满足银保监会监
管合规要求



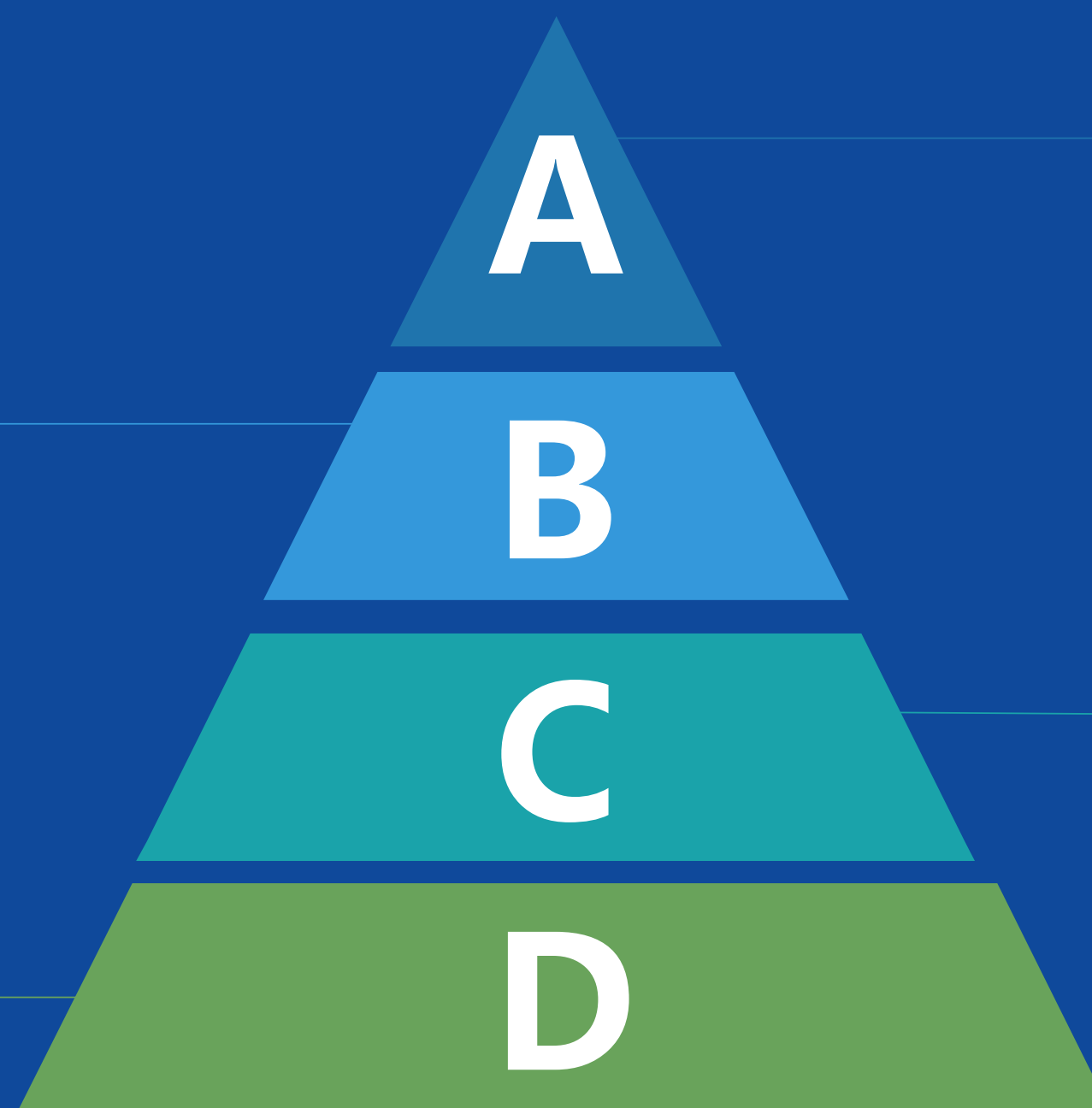
性能

高并发
高性能
扩展性

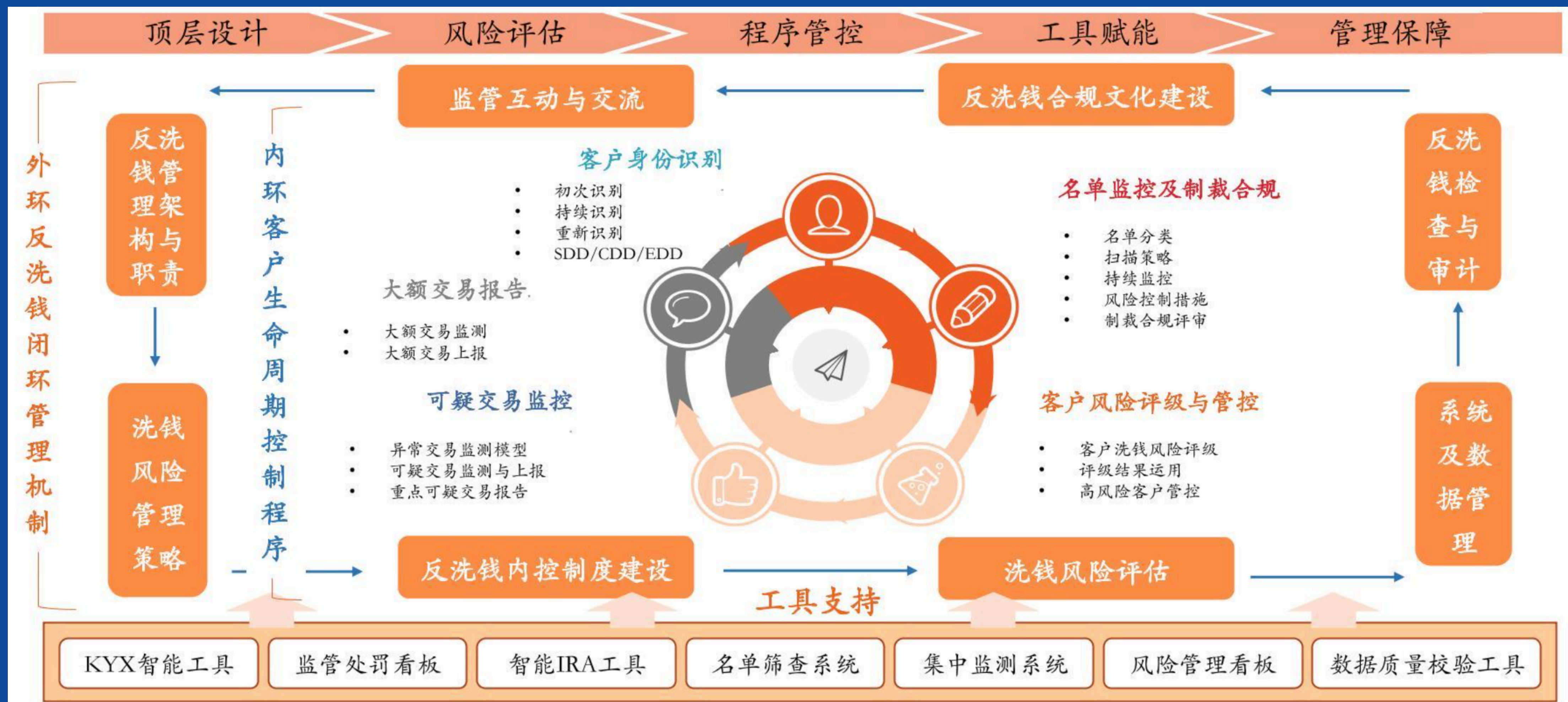


安全

物理安全
数据安全
网络安全
业务安全

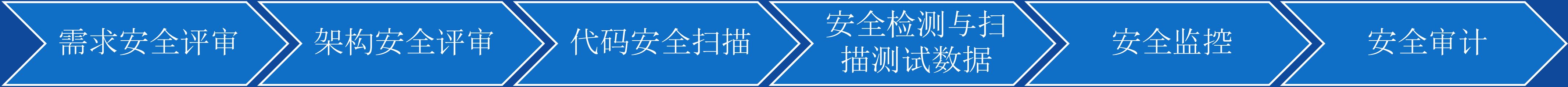


监管合规架构体系

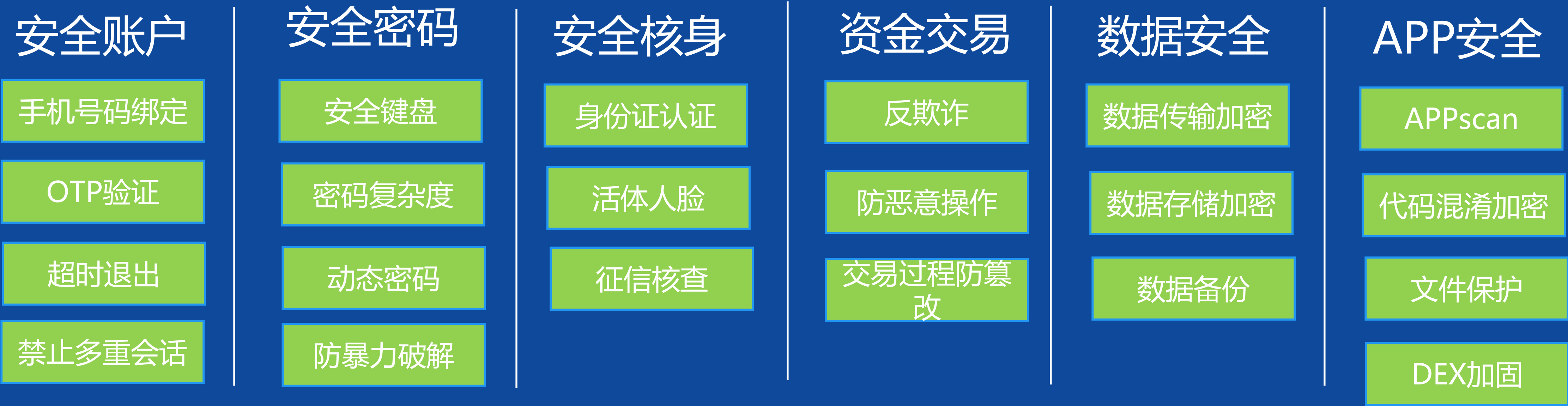


银行安全架构体系

研发过程



业务安全



物理安全

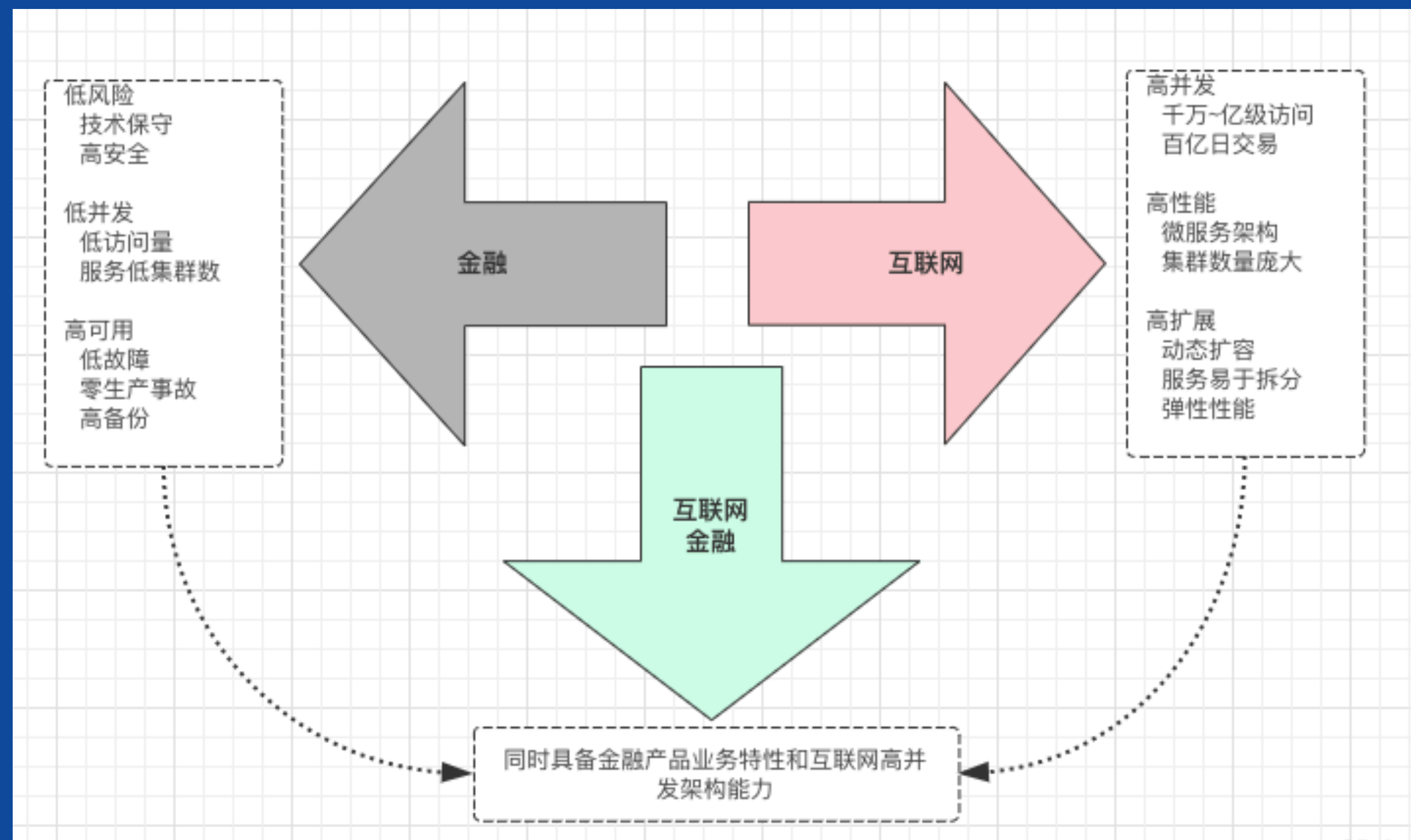


网络安全



互联网消费金融高并发架构面临的挑战

- 1. 对于架构师来说，前期的技术架构设计和领域规划，需要同时具备对**传统金融领域知识**和**互联网高并发架构**的双重能力；
- 2. 对于研发、测试、运维人员来说，系统的**复杂度**成倍的增长，微服务拆分后，系统研发、测试、运维难度大大增加；
- 3. 当出现线上问题时，排查问题和分析错误，也变得复杂，需要依赖庞大的**监控体系**和**log分析工具**。



如何应对稳定性与高并发挑战？

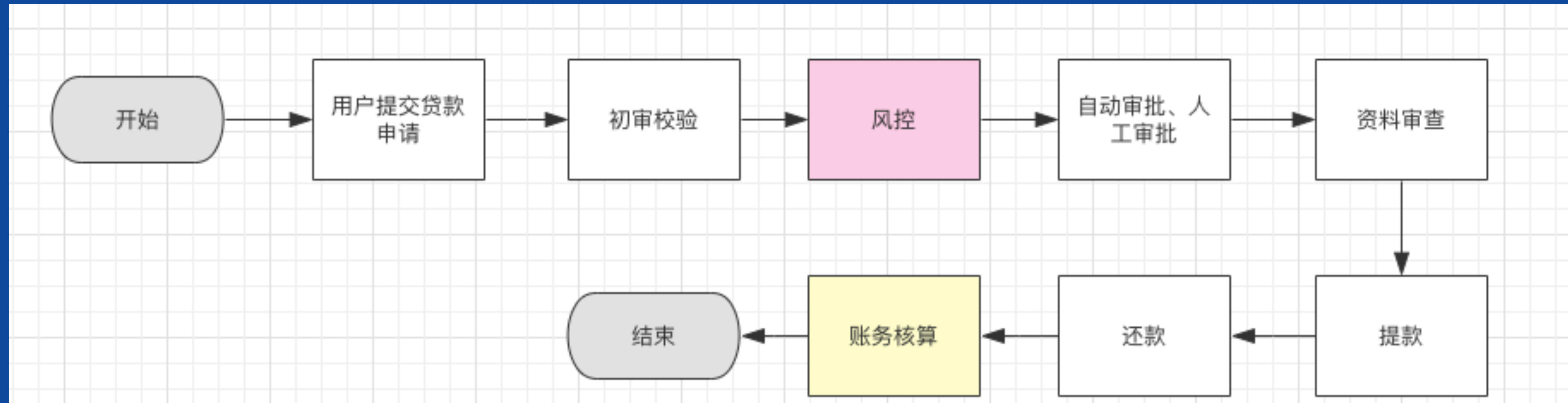


平安消费金融架构演进

大纲

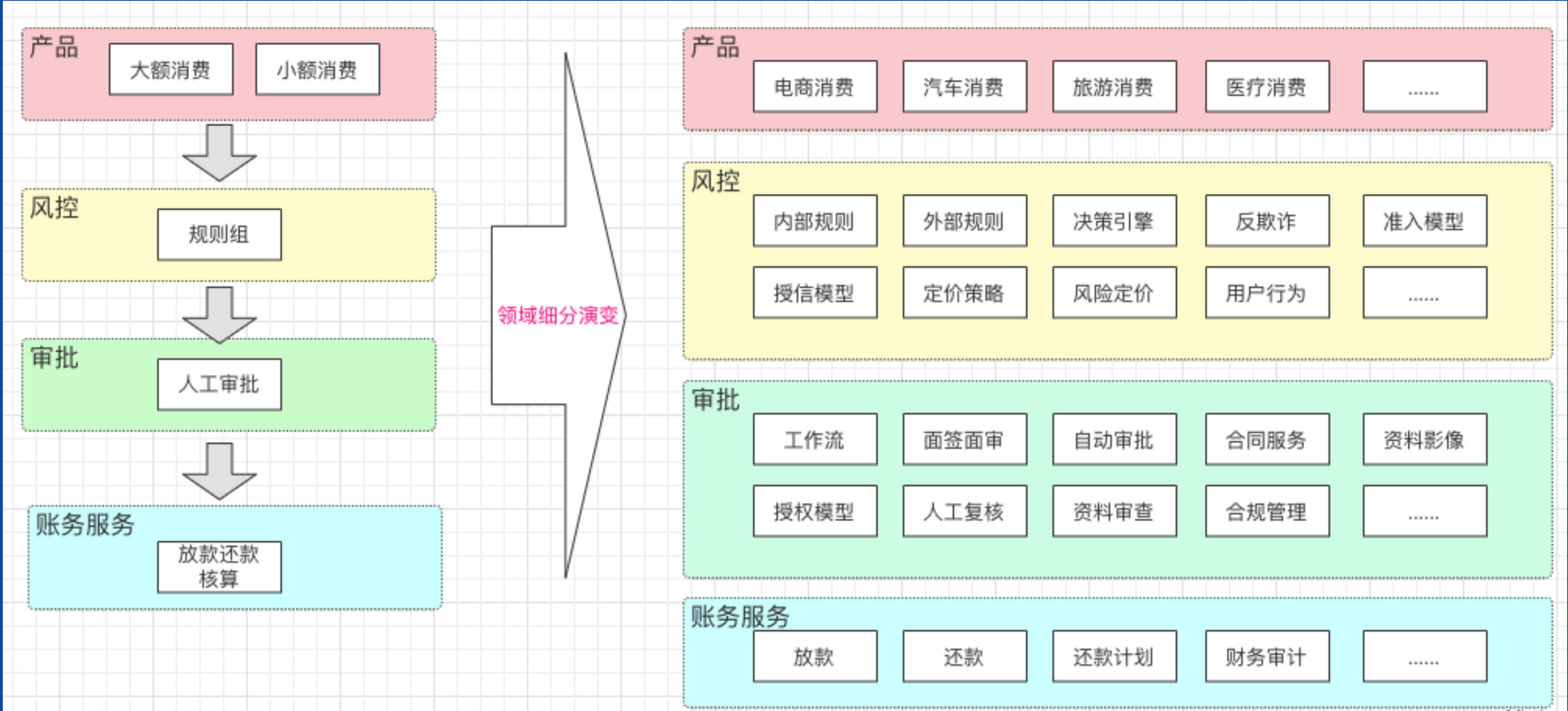
- 互联网消费金融架构的特性及面临的挑战
- 平安消费金融平台的服务化架构方案演进（单体-SOA-微服务）
- 平安消费金融平台微服务中间件选型及演进
- 平安消费金融平台架构演进面临的常见问题及思考

互联网消费金融贷款的生命周期



- 如果有逾期还涉及到贷后催收、核销等环节

平安消费金融的领域细分演变过程



平安微服务架构演进



银行金融架构演进需要解决的问题

金融系统耦合度高

金融业务涉及面广，但系统模块划分简单，并且**模块职责**不明确；
所有模块共用一个数据库，主数据库的表已经超过一千张，大量大表；

1

金融系统服务间调用链混乱

服务间的调用随意，没有进行合理的服务**调用链规划**；存在循环调用，调用链过长等问题；老系统往往缺乏**熔断、限流、降级**，超时控制，蓄洪等服务治理的能力；

2

金融系统性能差

系统臃肿导致系统容易出现大规模单点故障，**牵一发而动全身**；
单点的jvm和容器的**性能瓶颈**容易成为性能瓶颈；

3

快速交付的挑战

如果服务臃肿庞大，就难以做到快速灵活的发布；需要进行**微服务化**设计，将庞大臃肿的系统**化整为零**，每个服务集群能够独立发布和部署，快速应对性能需求和业务需求。

4

演进实践：服务化拆分

- 银行老安硕单体应用拆分、重构


拆分出应用200+

应用按**业务模块**，拆分成独立服务；

独立研发、独立代码仓库、各个服务均可独立发布和部署，独立运维；

每个服务可被多个应用**共享**；

服务之间可以通过HTTP、RPC、Soap等协议或MQ进行**通信**。

微服务拆分：稳定性设计

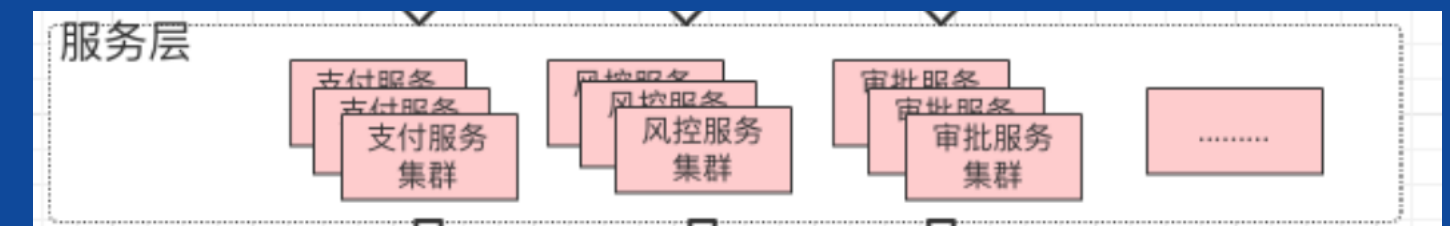
高可用HA (High Availability)

- * 不能“挂”
- * 可用性99.99%四个九
- * 一年故障时长0.876小时
- * 平均响应时间<100ms, 95线<500ms
- * 全年数据故障不超过5次
- * 全年系统100%可用

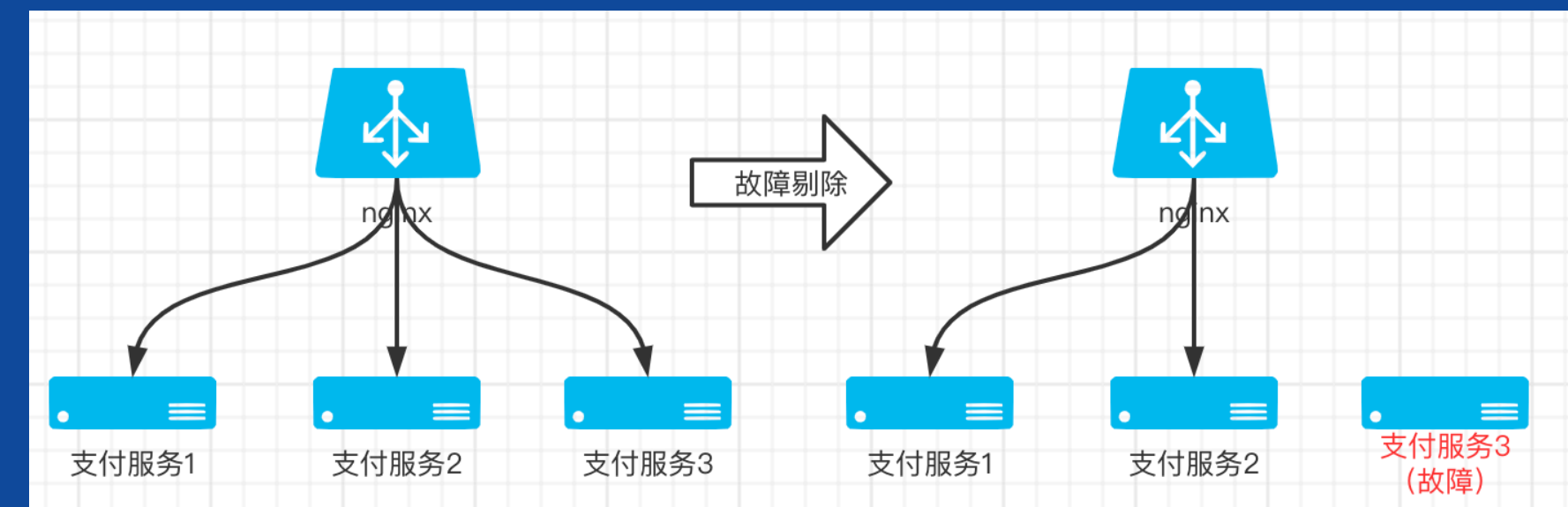
如何保证高可用？



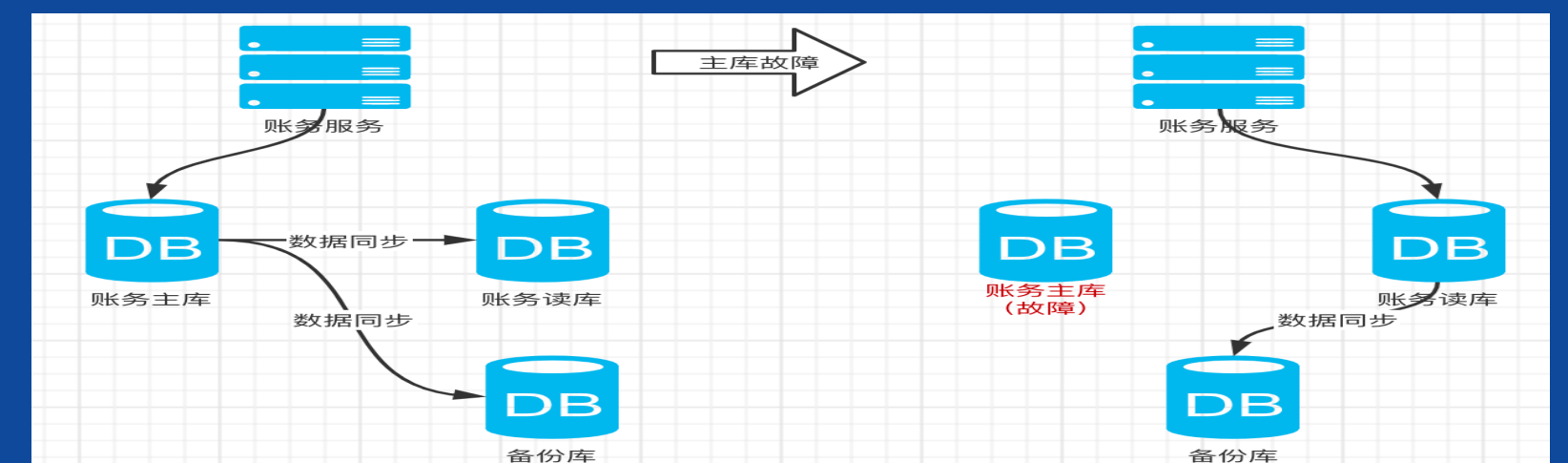
多副本设计



自动故障转移

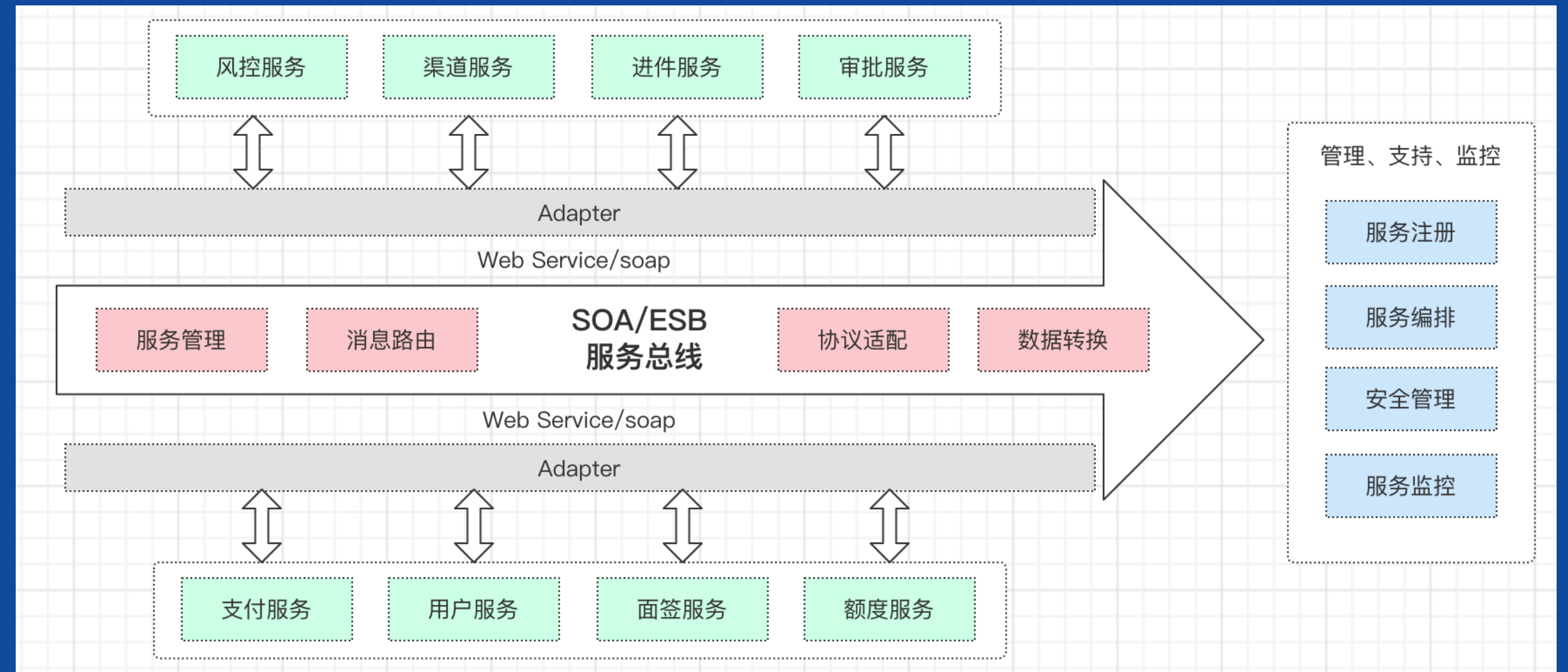


数据库：主备、分布式 (TiDB或者TDSQL)



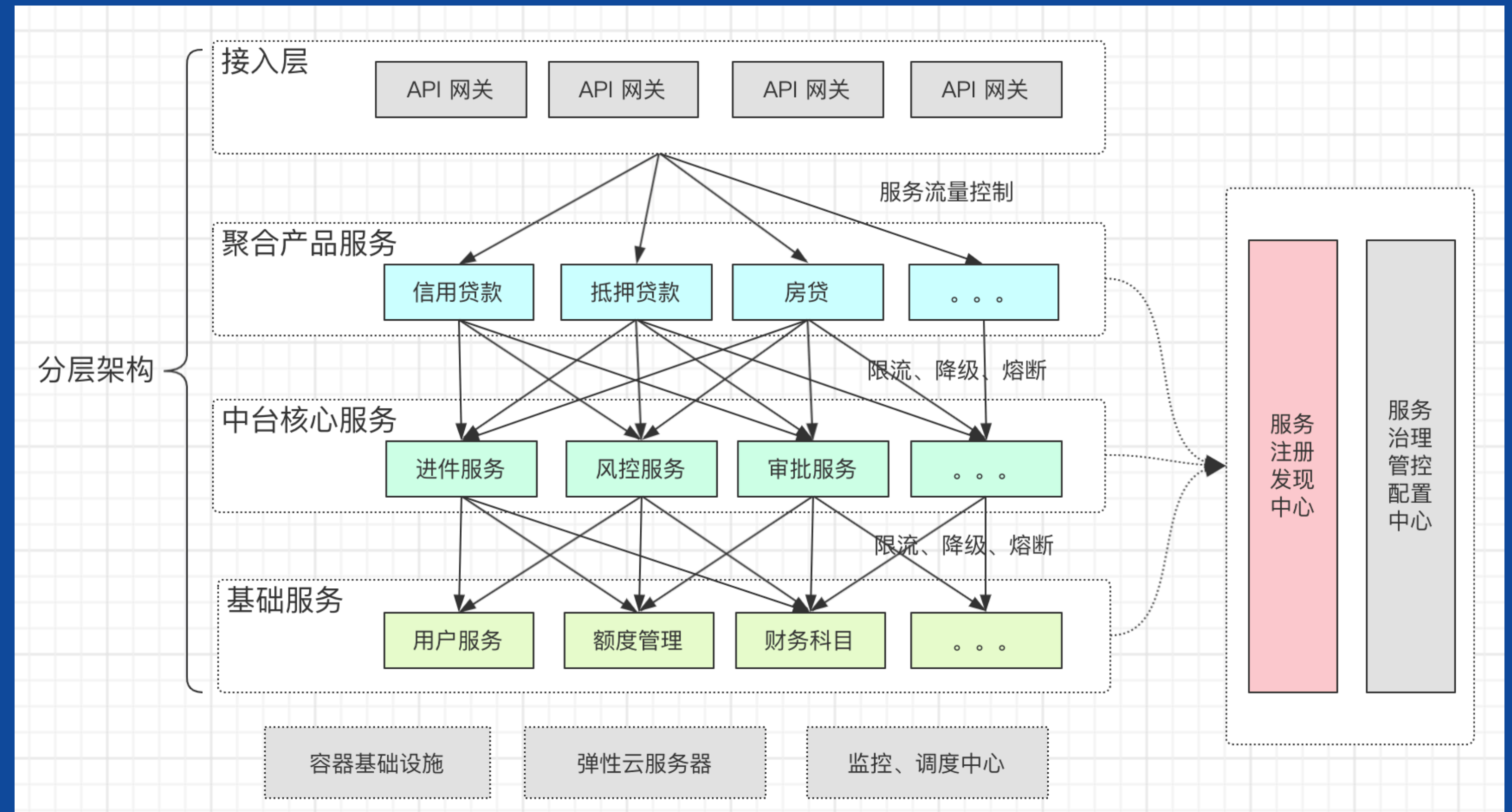
演进实践：SOA架构

- 主要解决的问题是银行海量已有系统的整合(互联互通)问题
- 手工治理比重大、自动化程度不足
- 技术实现及流程繁琐复杂、治理成本高
- 覆盖面广、涵盖企业IT各方面，和IT治理重叠度高
- 传统IT大厂(IBM、Oracle)把持标准



演进实践：微服务分层架构

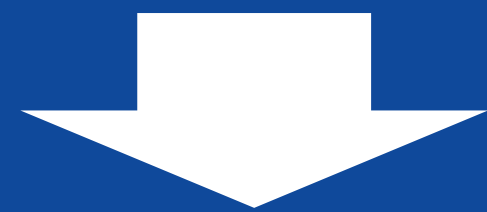
- ◆ 建设微服务架构，搭建**微服务治理平台**、**标准化**微服务设计，DDD；
- ◆ **量变导致质变**，不仅仅是服务化架构的延伸 **组织架构、管理策略、研发模式、测试、运维等领域** 都要做出相应的调整，以为微服务架构的落地创造合 适的“土壤”。
- ◆ 基于Dubbo/SpringCloud微服务架构二次开发**平安自研Pafa/Halo**平台线上化全生命周期的服务治理
- ◆ 测试**自动化**、运维**智能化**



有了微服务架构
我们还需要什么能力？

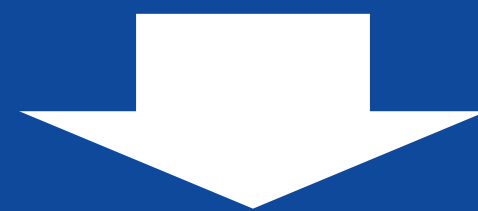
能力扩展

稳定性扩展



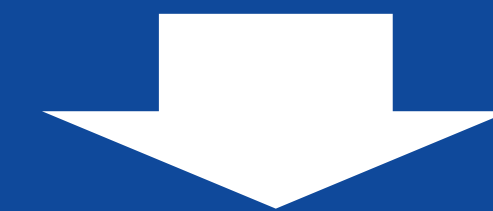
灰度蓝绿
发布能力建设

高可用扩展



同城双活
异地多活

可视化



监控体系建设

稳定的保障：灰度发布能力建设

- 首先选取**种子用户**，哪些群体用户能够进行灰度版本体验；



- 流量路由控制**：可以根据服务名(serviceName)、方法名(methodName)、版本号(versionName) 进行、ip 规则等进行流量路由。



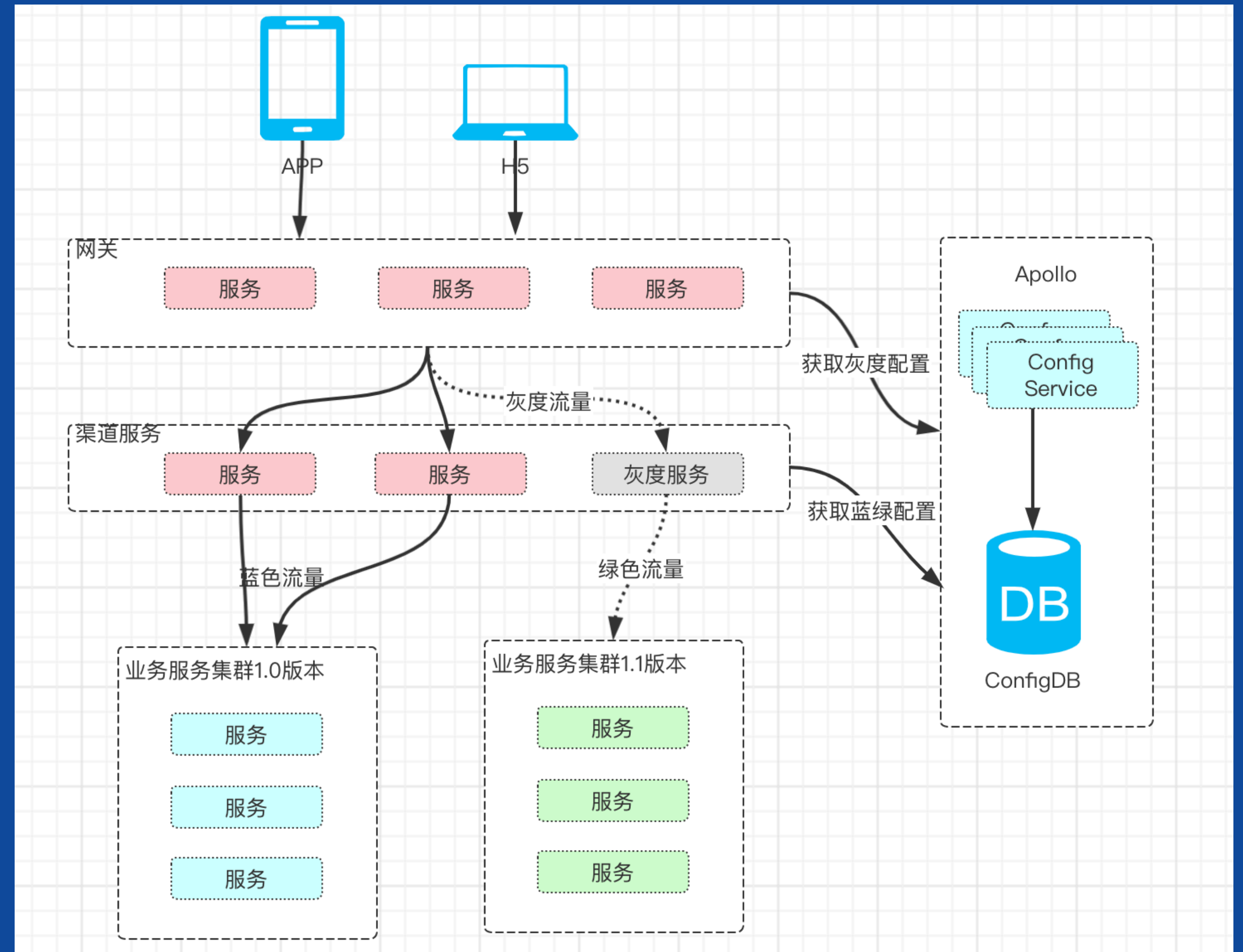
- 版本管理**：包括版本信息、升级地址、升级方案、是否全量发布。其中升级方案包括热更新及官网更新，全量发布以最新的全量版本为准；



- 服务部署**：每个服务集群管理一个版本，正式服务集群和灰度服务集群尽量配置和数量相等，也可以根据流量的多少进行动态分配。

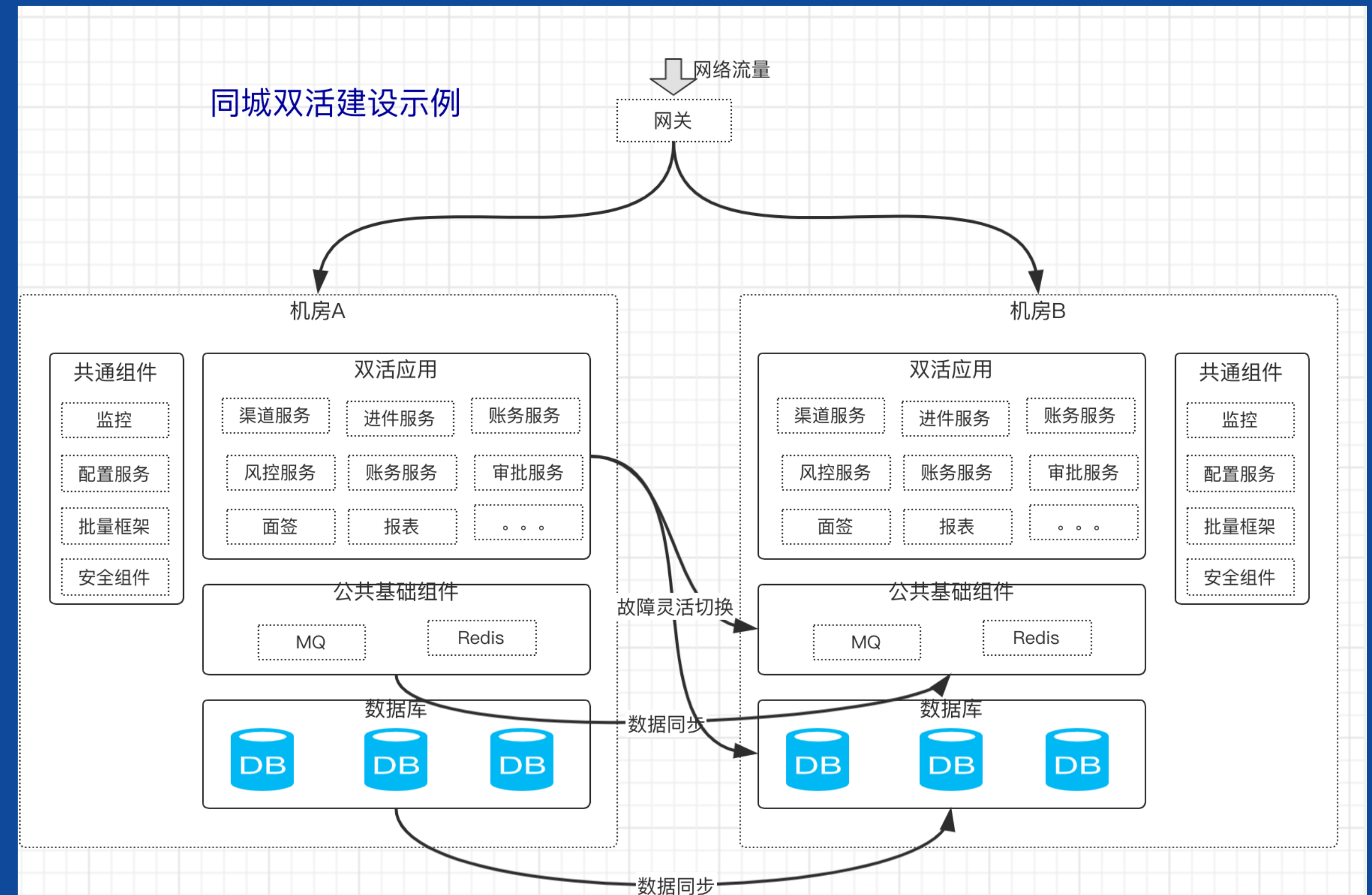


- 灰度验证**：将灰度流量逐步增加，需要同时验证业务功能的效果和系统架构的性能；验证完毕后可以考虑将所有集群统一升级至希望的版本



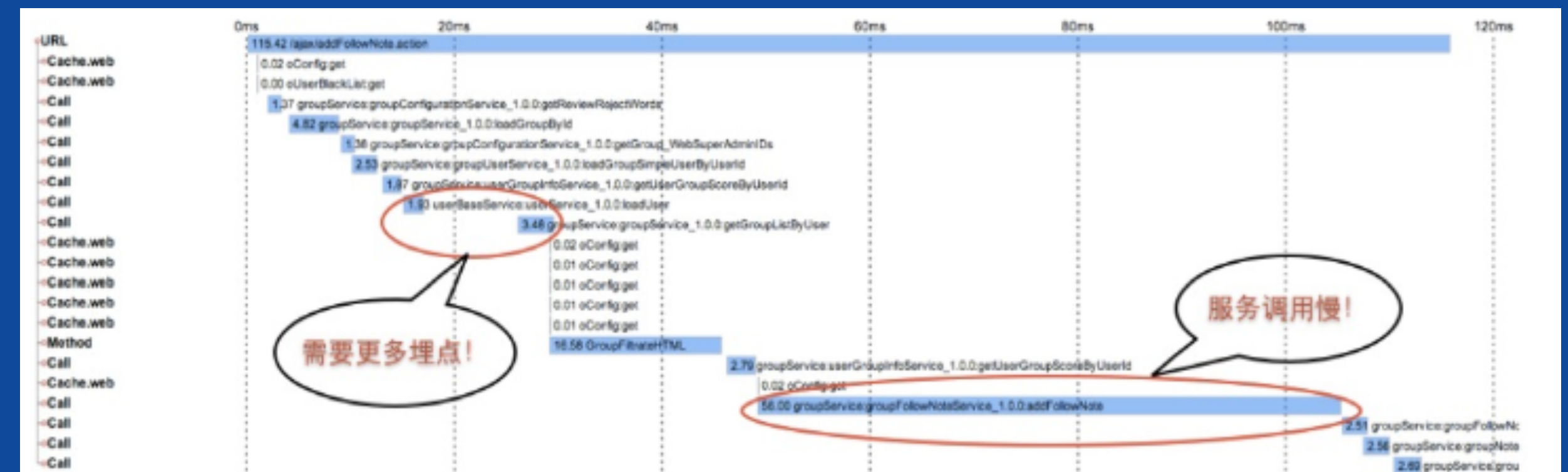
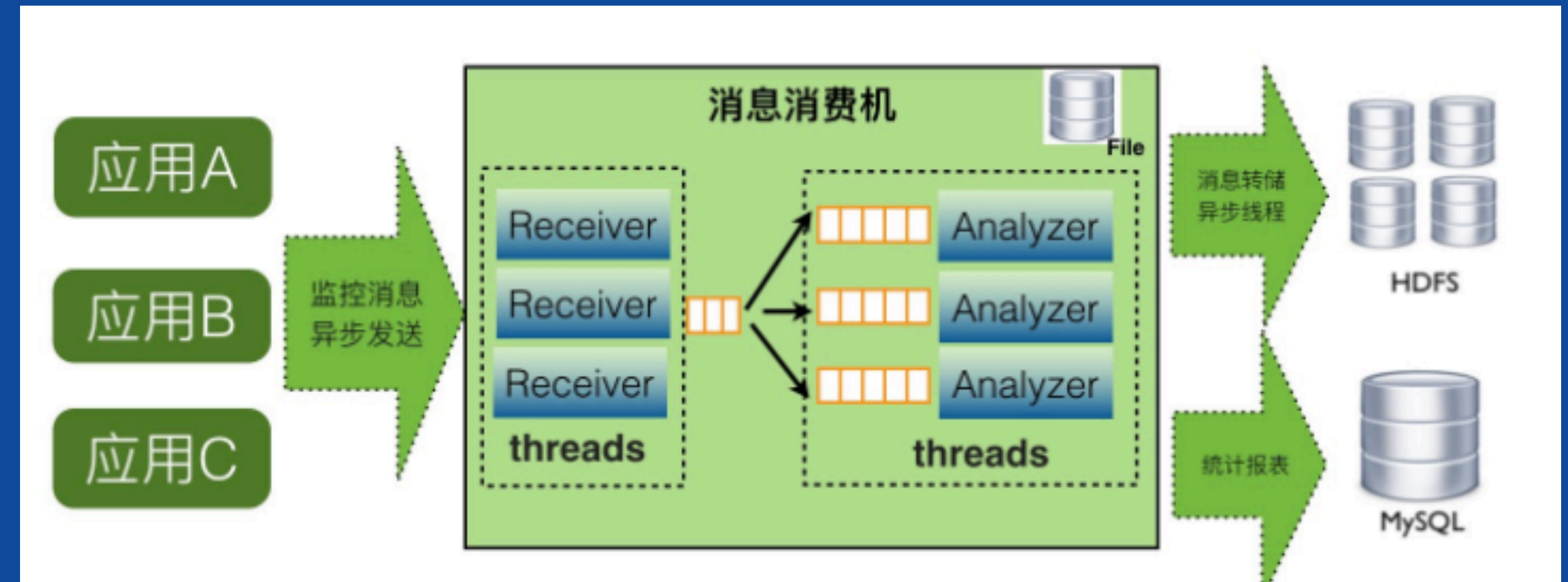
高可用：同城双活、异地多活

- 在银行、金融行业，数据和服务的**高可用**重要程度都非常高
- 通常会通过建设同城双活、异地多活的架构，来提升系统的**容错和伸缩能力**



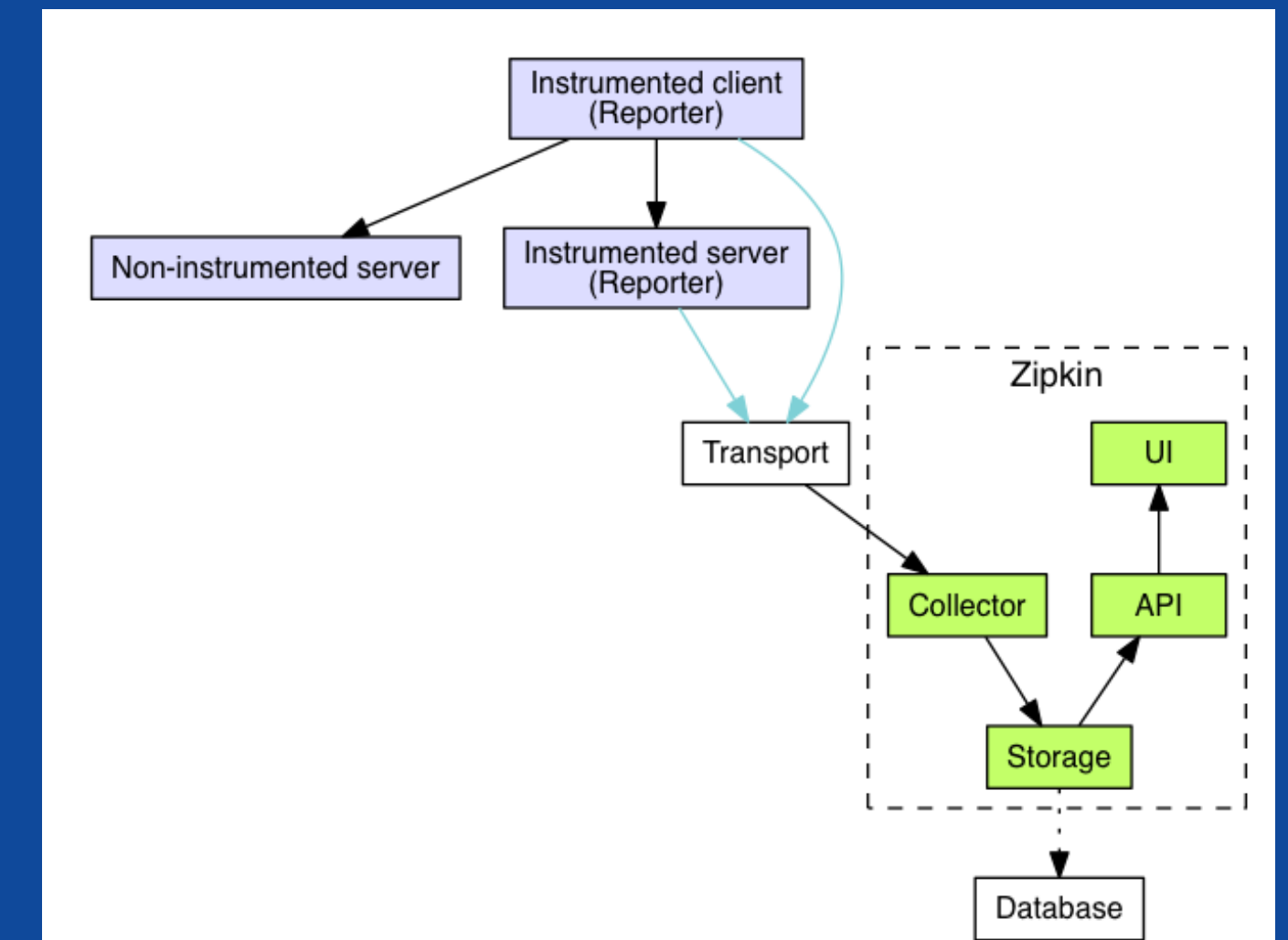
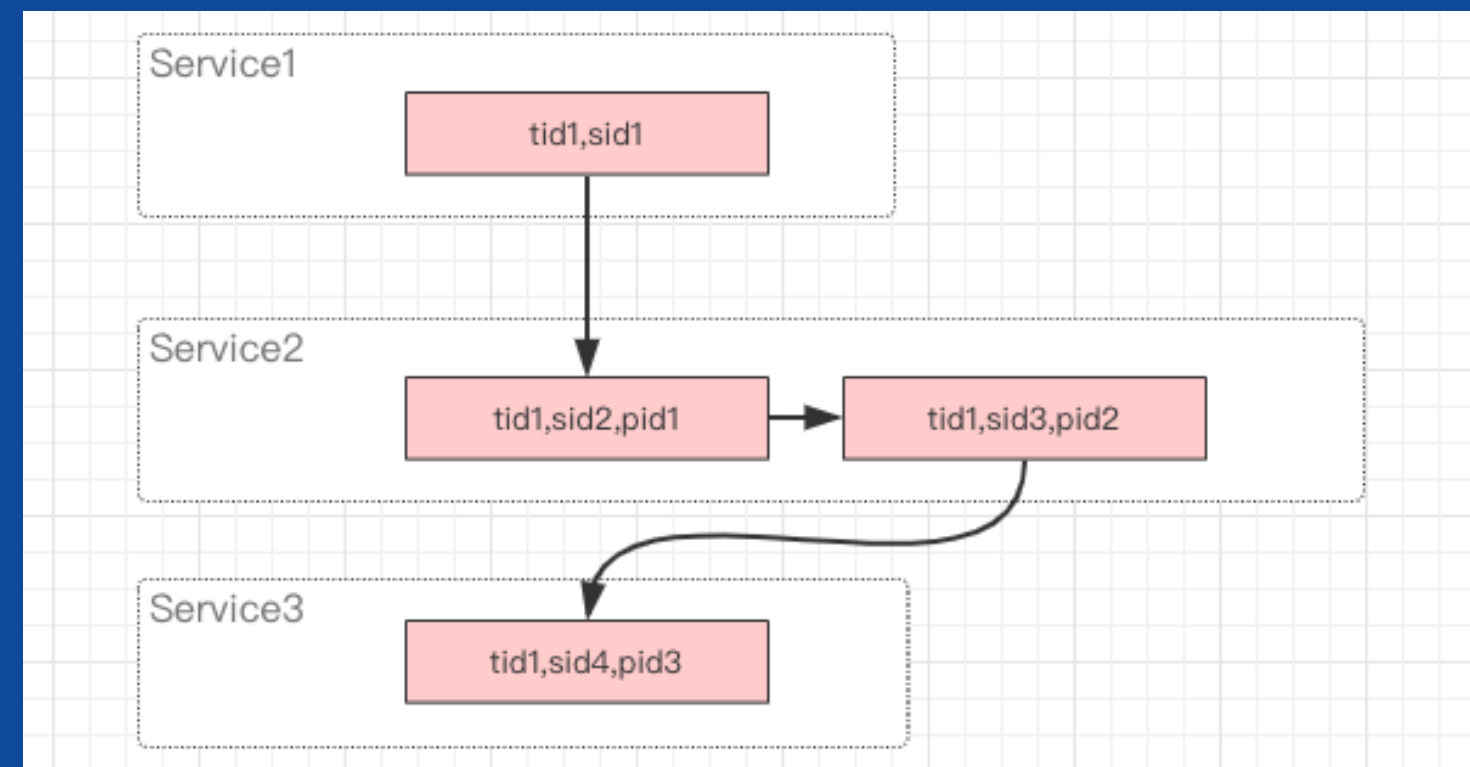
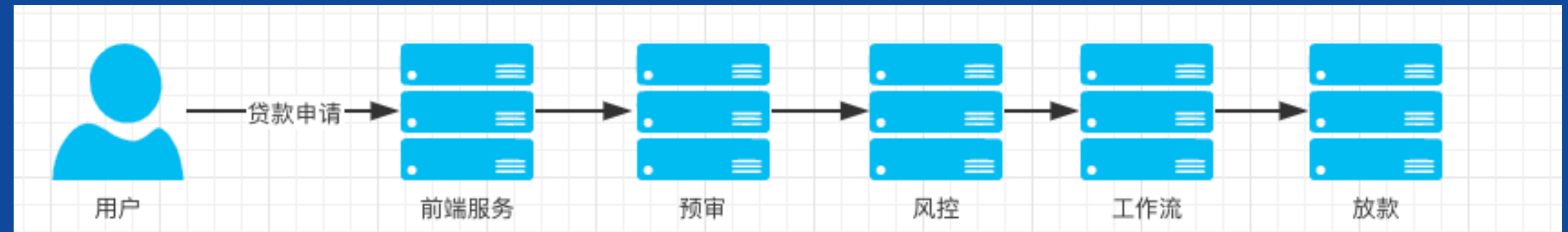
监控体系实践-CAT

- **实时处理**：信息的价值会随时间锐减，尤其是事故处理过程中
- **全量数据**：全量采集指标数据，便于深度分析故障案例
- **高可用**：故障的还原与问题定位，需要高可用监控来支撑
故障容忍：故障不影响业务正常运转、对业务透明
- **高吞吐**：海量监控数据的收集，需要高吞吐能力做保证
- **可扩展**：支持分布式、跨 IDC 部署，横向扩展的监控系统



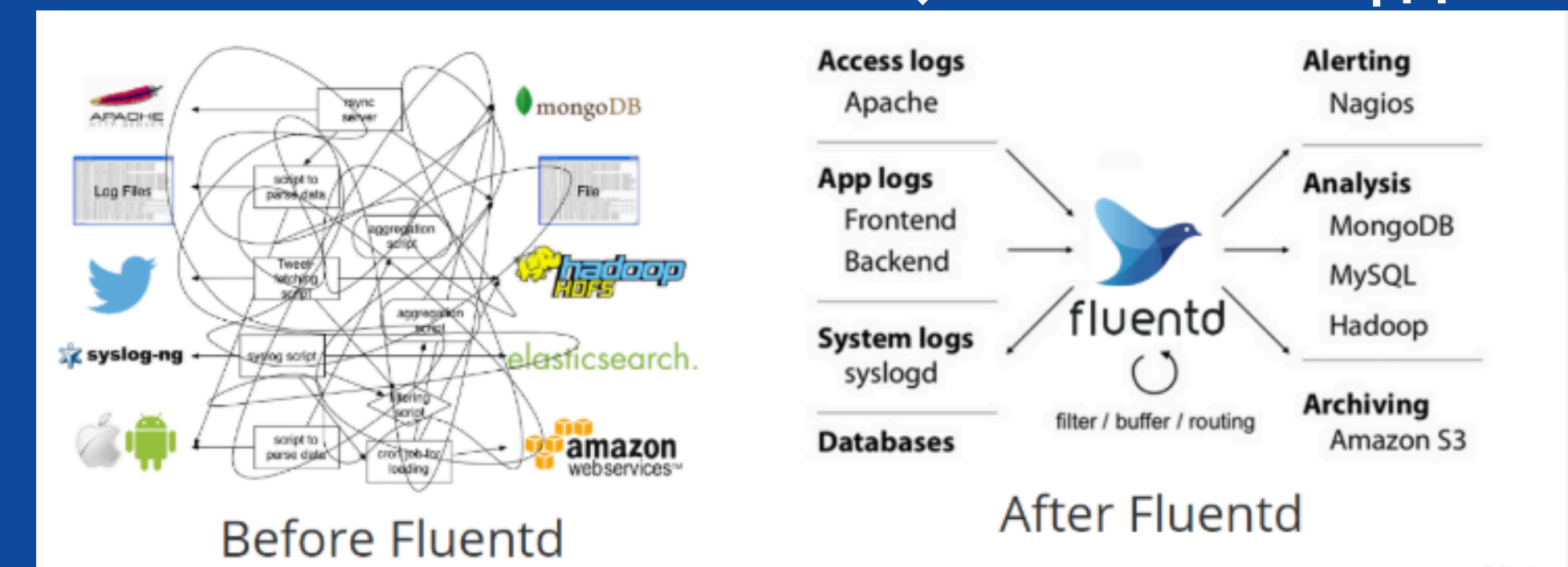
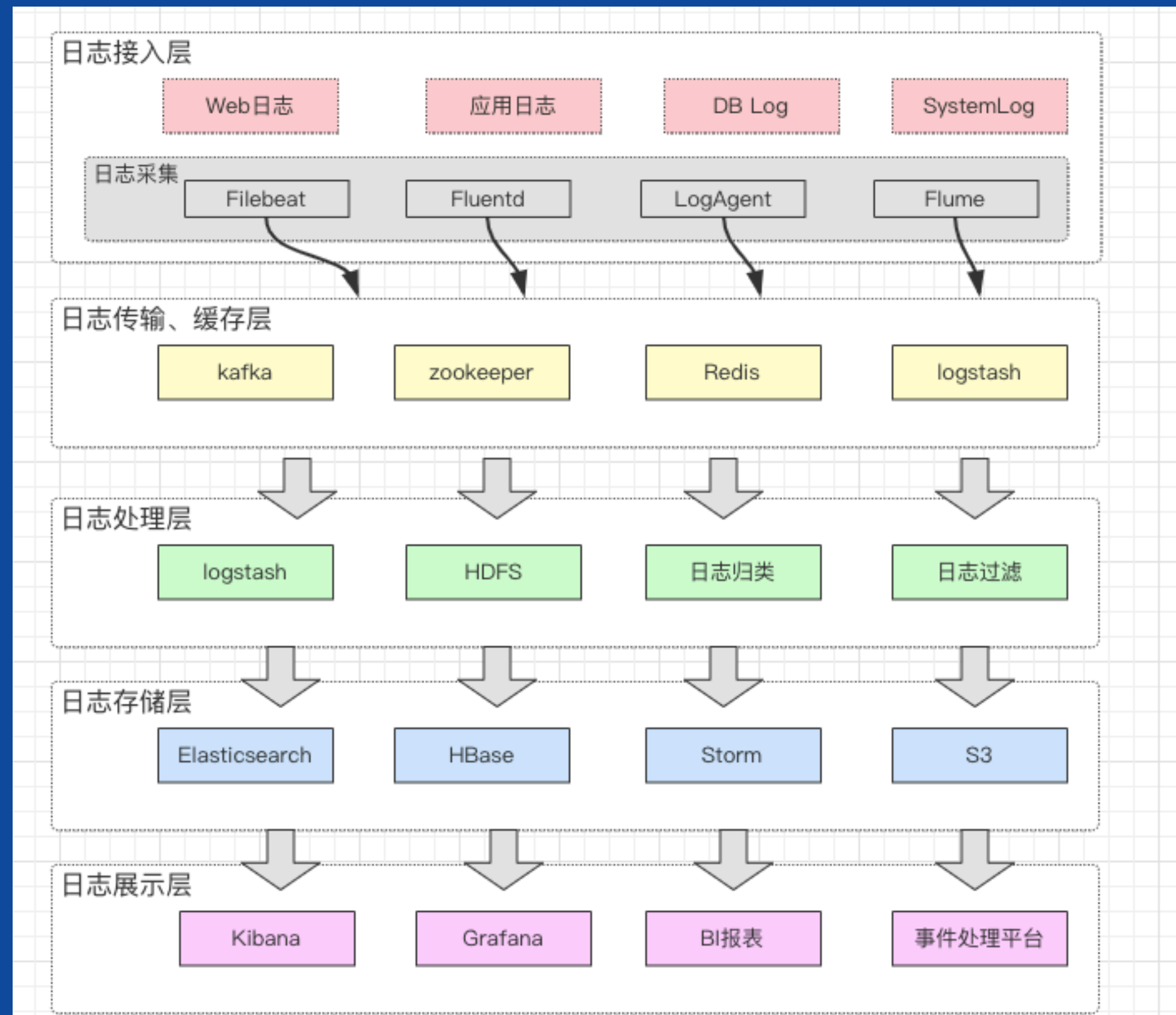
监控体系-调用链

- 统一TraceID埋点
- 建设应用层底层封装FrameworkFoundation.jar
- 协议支持RPC、HTTP、ESB等



监控体系实践-日志采集与监控大盘

EFK : Elasticsearch、Fluentd和Kibana



监控大盘



实践：业务监控

埋点

- 事务Transaction
- 事件Event
- 心跳Heartbeat
- 业务指标Metric

监控

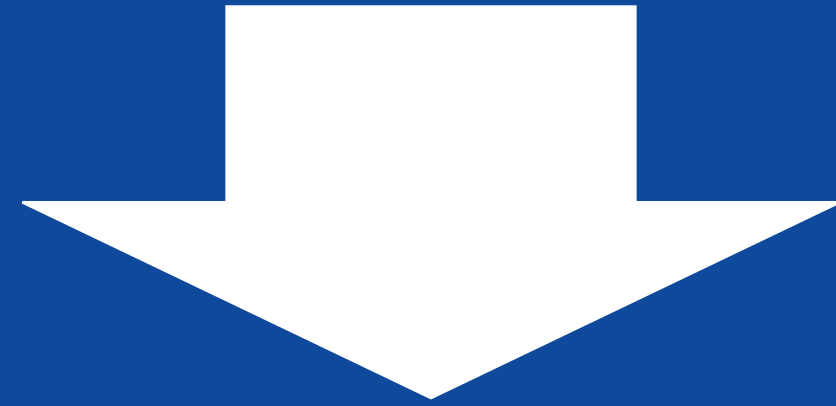
- 业务汇总类监控
- 贷款流程监控
- 定时跑批监控
- 外部接口监控

大盘、告警



Type	Total 总数	Failure 错误数	Failure% 失败率	Sample Link 样例链路	Min(ms) 最小耗时	Max(ms) 最大耗时	Avg(ms) 平均耗时	95Line(ms) 95线	99.9Line(ms) 99.9线	Std(ms)	QPS
[:: show ::] URLForward	11	1	9.0909%	Log View	0	2,207	367.6	2,000.0	2,000.0	607.3	0.0
[:: show ::] URL	43	1	2.3256%	Log View	0	2,475	108.3	550.0	2,000.0	386.3	0.1
[:: show ::] MVC	134	1	0.7463%	Log View	0	2,276	32.7	160.0	2,000.0	208.3	0.2
[:: show ::] ModelService	56	0	0.0000%	Log View	4	51	14.0	43.6	43.6	10.6	0.1
[:: show ::] CleanUpTransactionReports	7	0	0.0000%	Log View	3	10	5.1	10.0	10.0	2.1	0.0
[:: show ::] CleanUpTransaction	7	0	0.0000%	Log View	3	10	5.0	10.0	10.0	2.1	0.0
[:: show ::] 6. 点击展示单个Type的指标图表	161	0	0.0000%	Log View	0	575	4.5	2.1	512.7	45.1	0.2
[:: show ::] System	17	0	0.0000%	Log View	0	28	3.9	25.0	25.0	6.5	0.0
[:: show ::] TimerSync	112	0	0.0000%	Log View	0	57	3.4	20.0	55.0	7.9	0.1
[:: show ::] CleanUpEvent	7	0	0.0000%	Log View	0	1	0.1	1.0	1.0	0.3	0.0
[:: show ::] CleanUpEventReports	7	0	0.0000%	Log View	0	1	0.1	1.0	1.0	0.3	0.0
[:: show ::] Restore	20	0	0.0000%	Log View	0	1	0.1	1.0	1.0	0.3	0.0

微服务架构体系怎样落地？



平安银行中间选型与演进

大纲

- 互联网消费金融架构的特性及面临的挑战
- 平安消费金融平台的服务化架构方案演进（单体–SOA–微服务）
- **平安消费金融平台微服务中间件选型及演进**
- 平安消费金融平台架构演进面临的常见问题及思考

银行中间件架构的挑战

- 业务高速发展，流量和数据量**大增**；
- 在系统稳定性、可用性、扩展性、安全性和成本等面临挑战；

- 快速上线、**快速**交付能力（Time To Market），交付时间面临挑战；
- 管理结构BU化，千人研发人员**并行开发**，系统交付的质量面临挑战；

- 分布式的系统，复杂度高，调用链长，超出个人的处理能力，**工具化**势在必行；
- 多数据中心的部署，大量机器和应用服务面临**治理**的挑战；

- **编程语言**：Java，NodeJs，Python，GO等
- **数据库**：Oracle，MySQL，PostgreSQL，MongoDB等
- **中间件**：Redis，RocketMQ，Apollo等

系统
稳定性

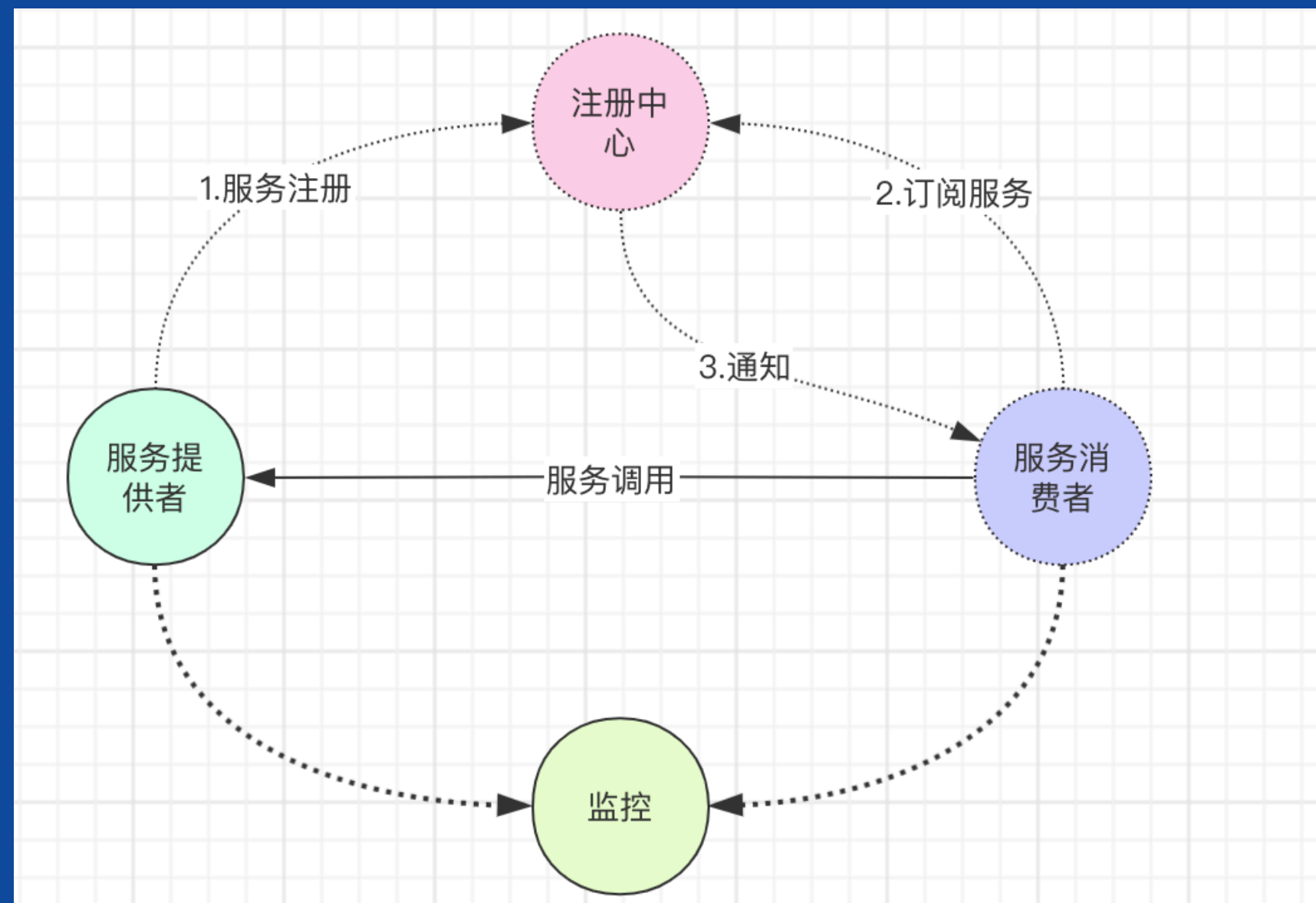
交付
能力

治理
效率

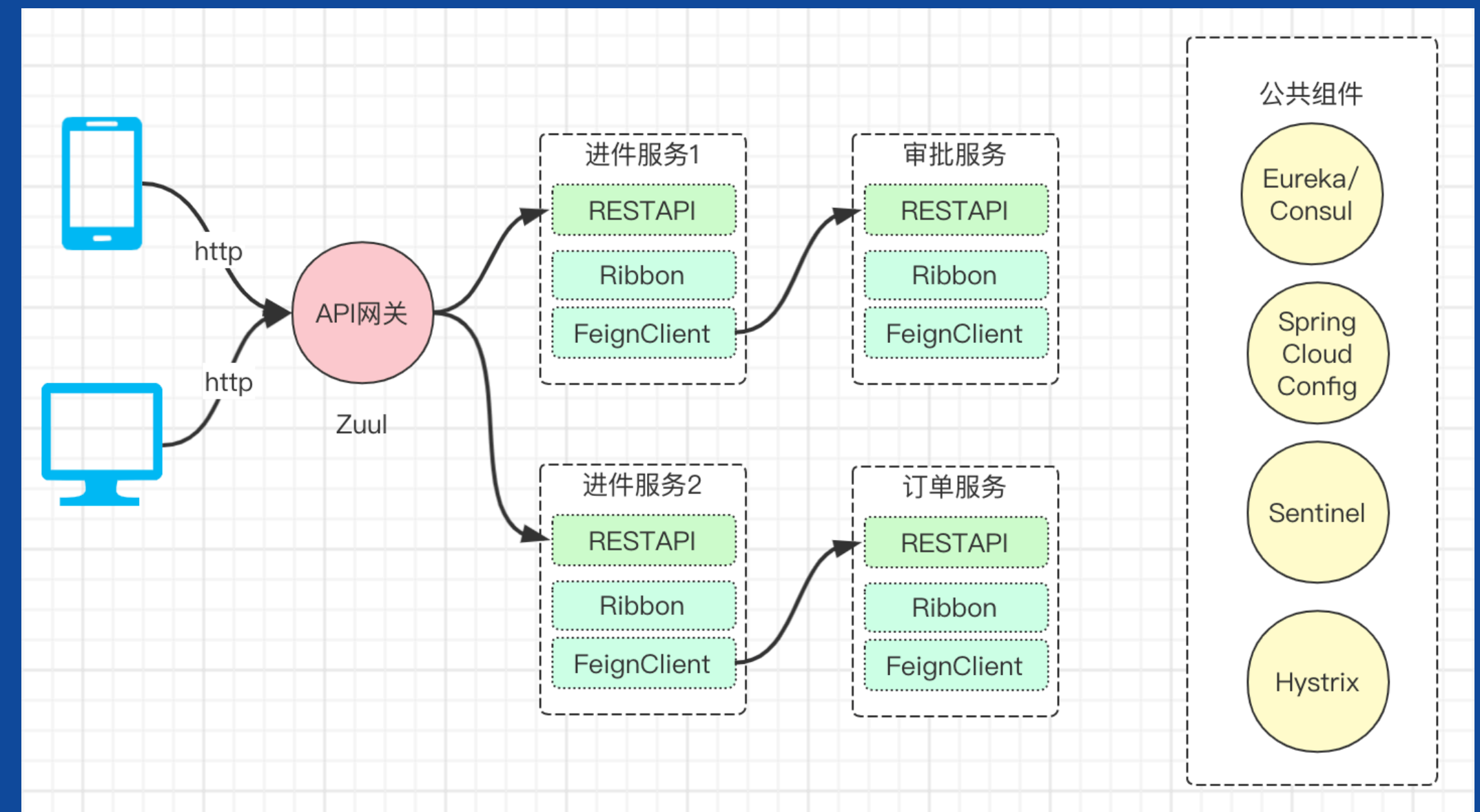
技术栈
规划

实战：微服务架构选型

- Dubbo（简单、快）
- 自研Pafa平台（zk、RPC、pizza）



演进



- Spring Cloud（生态体系全）
- 自研Halo平台（Consul、Ribbon、FeignClient、Gateway、Hystrix）

实践：建设服务注册发现中心

技术选型

Zookeeper

- 社区活跃度：中
- CAP模型：CP
- 控制台管理：不支持
- 适用规模（建议）：十万级
- 健康检查：Keep Alive
- 易用性：易用性比较差，Zookeeper的客户端使用比较复杂，没有针对服务发现的模型设计以及相应的API封装，需要依赖方自己处理。对多语言的支持也不太好，同时没有比较好用的控制台进行运维管理。
- 综合建议：更新较慢，功能匮乏，使用部署较复杂，不易上手，维护成本较高。

Eureka

- 社区活跃度：低，已停止开源维护
- CAP模型：AP
- 控制台管理：支持
- 适用规模（建议）：十万级
- 健康检查：Client Beat
- 易用性：较好，基于SpringCloud体系的starter，帮助用户以非常低的成本无感知的做到服务注册与发现。提供官方的控制台来查询服务注册情况。
- 综合建议：eureka当前停止开源不建议企业级使用

Consul（采用）

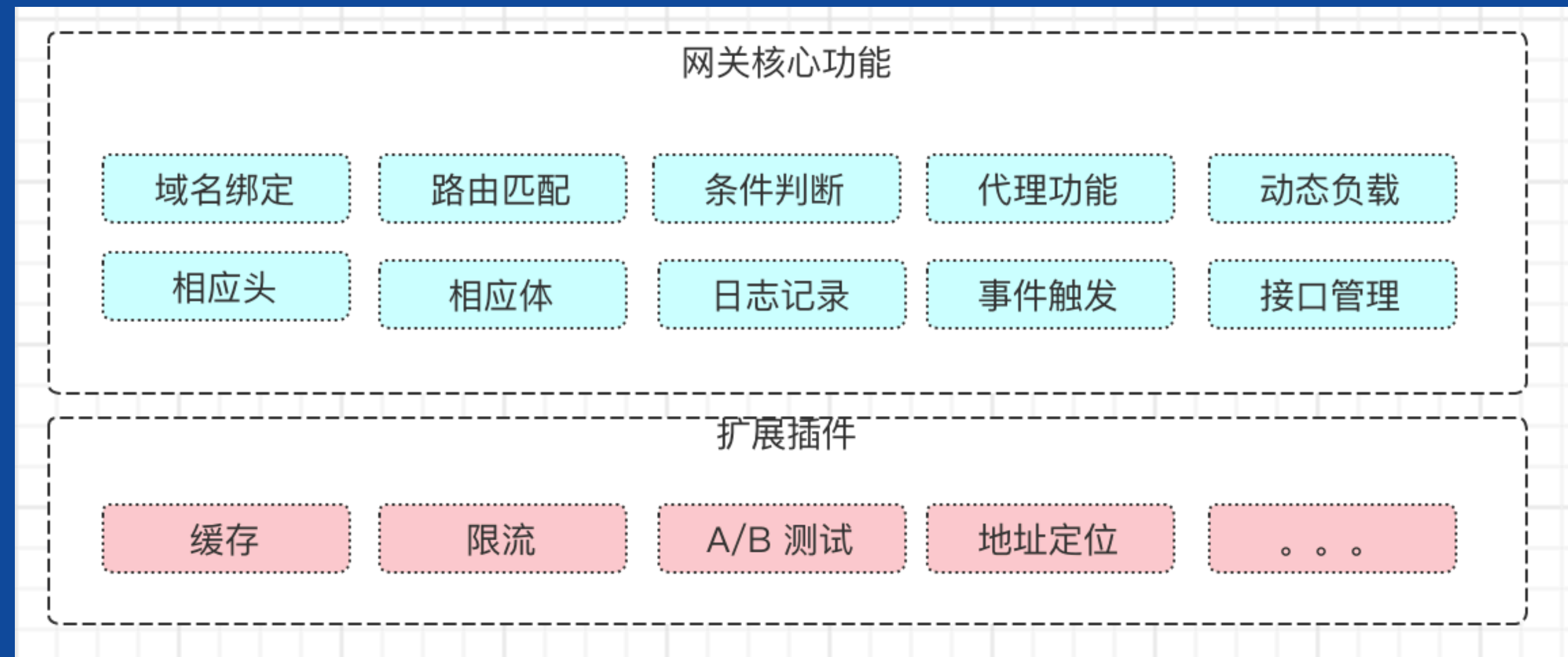
- 社区活跃度：高
- CAP模型：CP
- 控制台管理：支持
- 适用规模（建议）：百万级
- 健康检查：TCP/HTTP/gRPC/Command
- 易用性：较好，能够帮助用户以非常低的成本无感知的做到服务注册与发现。提供官方的控制台来查询服务注册情况。
- 综合建议：集成简单，不依赖其他工具，推荐大中型企业使用。

Nacos

- 社区活跃度：高
- CAP模型：CP+AP
- 控制台管理：支持
- 适用规模（建议）：百万级
- 健康检查：TCP/HTTP/MYSQL/Client Beat
- 易用性：较好，能够帮助用户以非常低的成本无感知的做到服务注册与发现。提供官方的控制台来查询服务注册情况。
- 综合建议：阿里巴巴背书，更新速度快，文档完善，社区活跃度高，推荐大中型企业使用。

基于OpenResty打造高性能网关

- 1. 智能路由
- 2. 业务隔离
- 3. 熔断限流
- 4. 动态更新
- 5. 灰度发布
- 6. 监控告警



OpenResty是一个基于 Nginx 与 Lua 的高性能 Web 平台，其内部集成了大量精良的 Lua 库、第三方模块以及大多数的依赖项。用于方便地搭建能够处理超高并发、扩展性极高的动态 Web 应用、Web 服务和动态网关。

实践：流量控制与调度

技术选型

Hystrix

- Spring Cloud官方默认的熔断组件Hystrix（已停止维护）；

resilience4j

- 较轻量的熔断降级库resilience4j（轻量级）；

RateLimiter

- Google开源工具包Guava提供了限流工具类RateLimiter（功能较单一）；

Sentinel（采用）

- 阿里巴巴的开源框架Sentinel（推荐）；

实践：流量控制与调度

基于Sentinel 的流量治理

轻量级

- 核心库无多余依赖
- 性能损耗小

方便接入

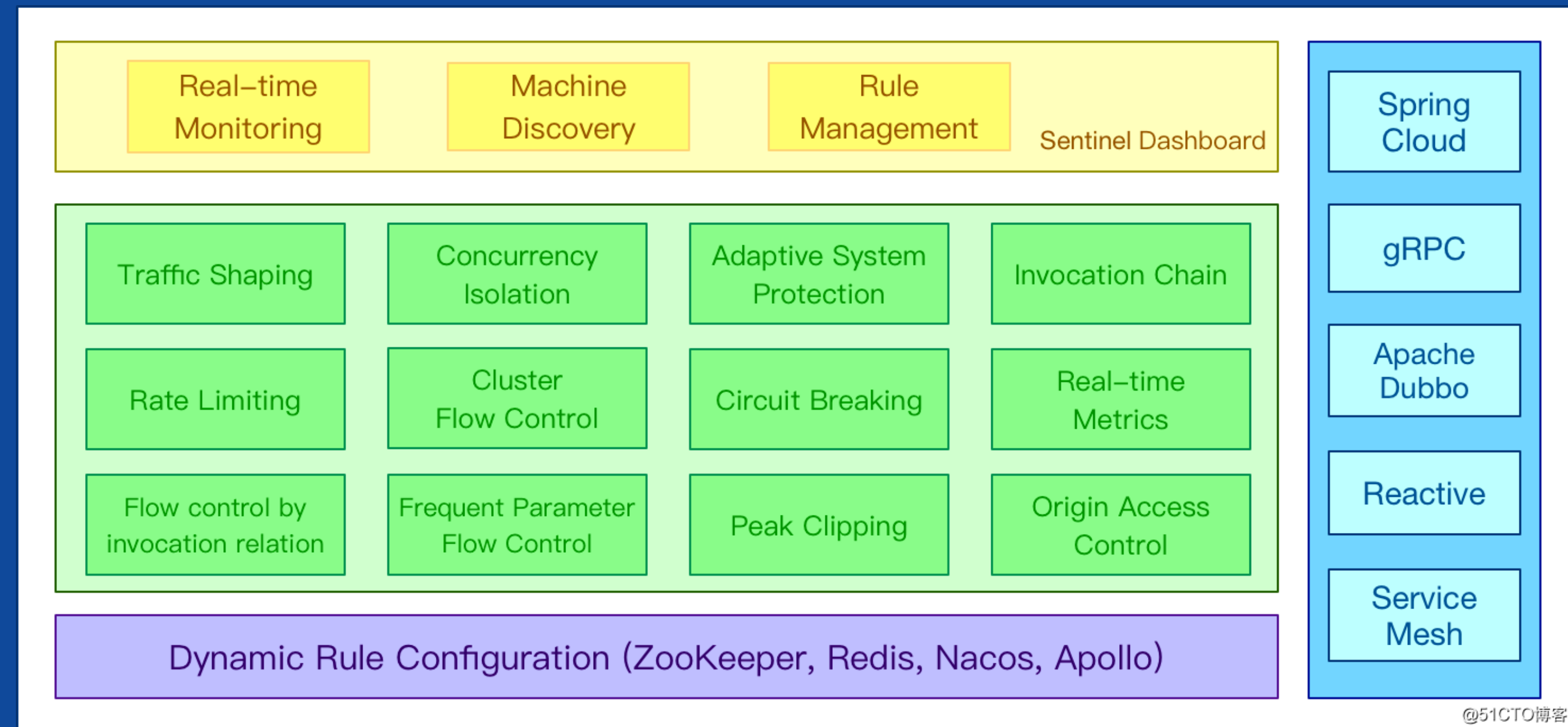
- Dubbo、Spring Cloud、Web Servlet、gRPC
- 自定义

流量控制

- 流控指标、流控效果（塑形）、调用关系、热点、集群等各种维度

控制台

- 实时监控、机器发现、规则管理等能力



@51CTO博客

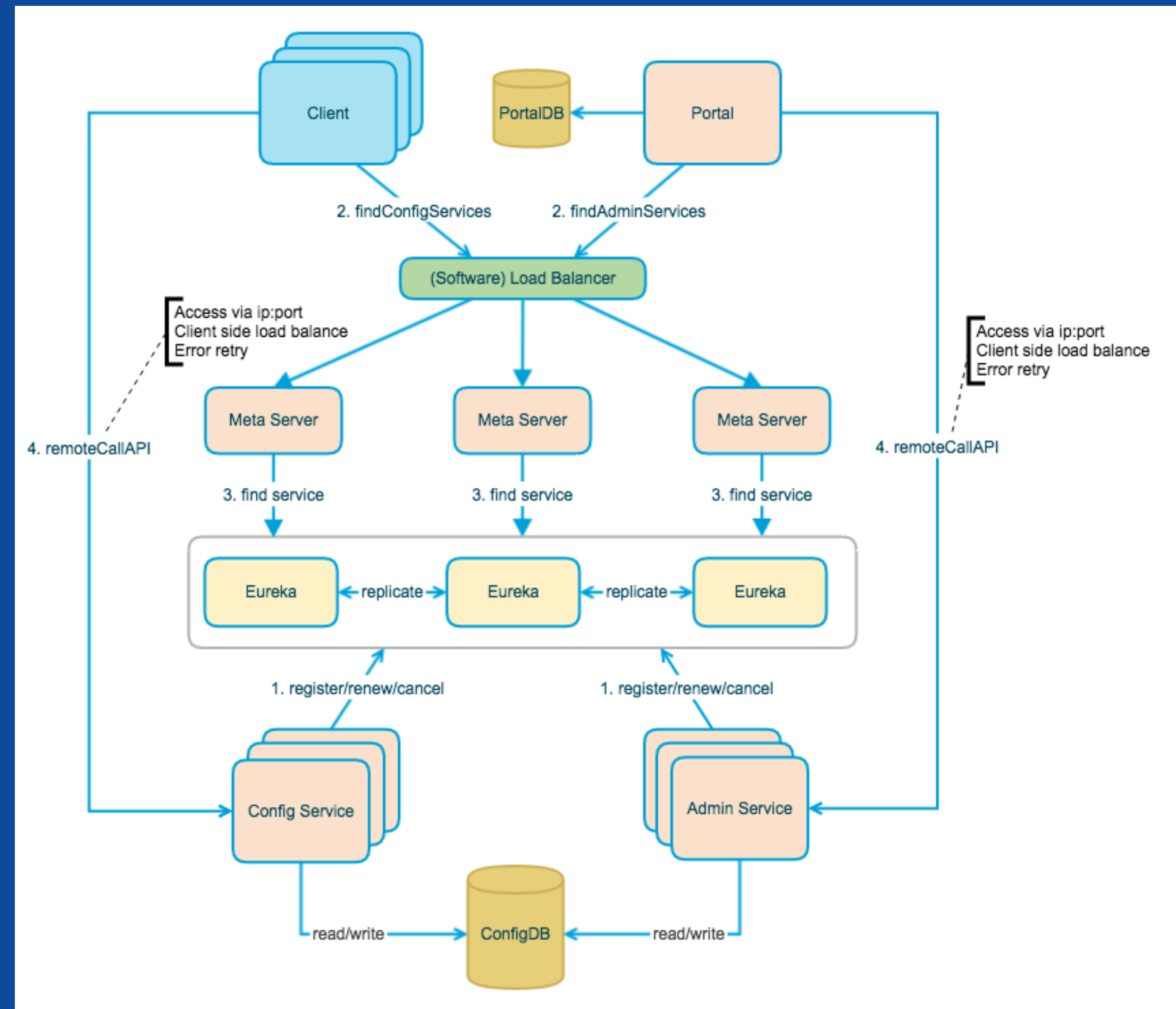
基于Apollo打造分布式配置中心

本地配置

演进

分布式配置

- 多环境多**集群**配置
- 配置修改**实时生效**
- **版本**发布管理
- 支持**灰度**发布
- 支持**权限/审核/审计**管理
- 开放**API**管理



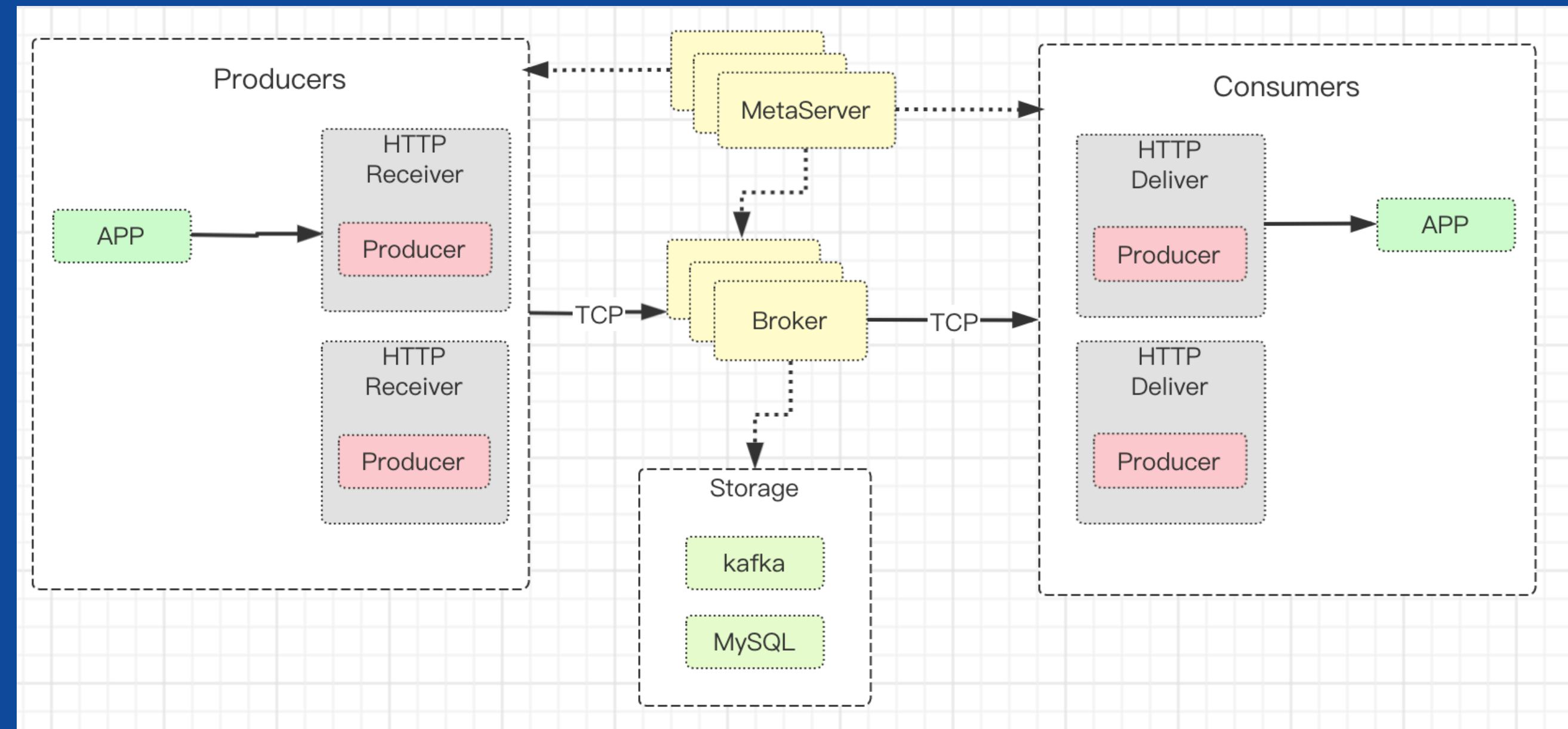
实践：消息队列中间件

银行传统IBM ISC

演进

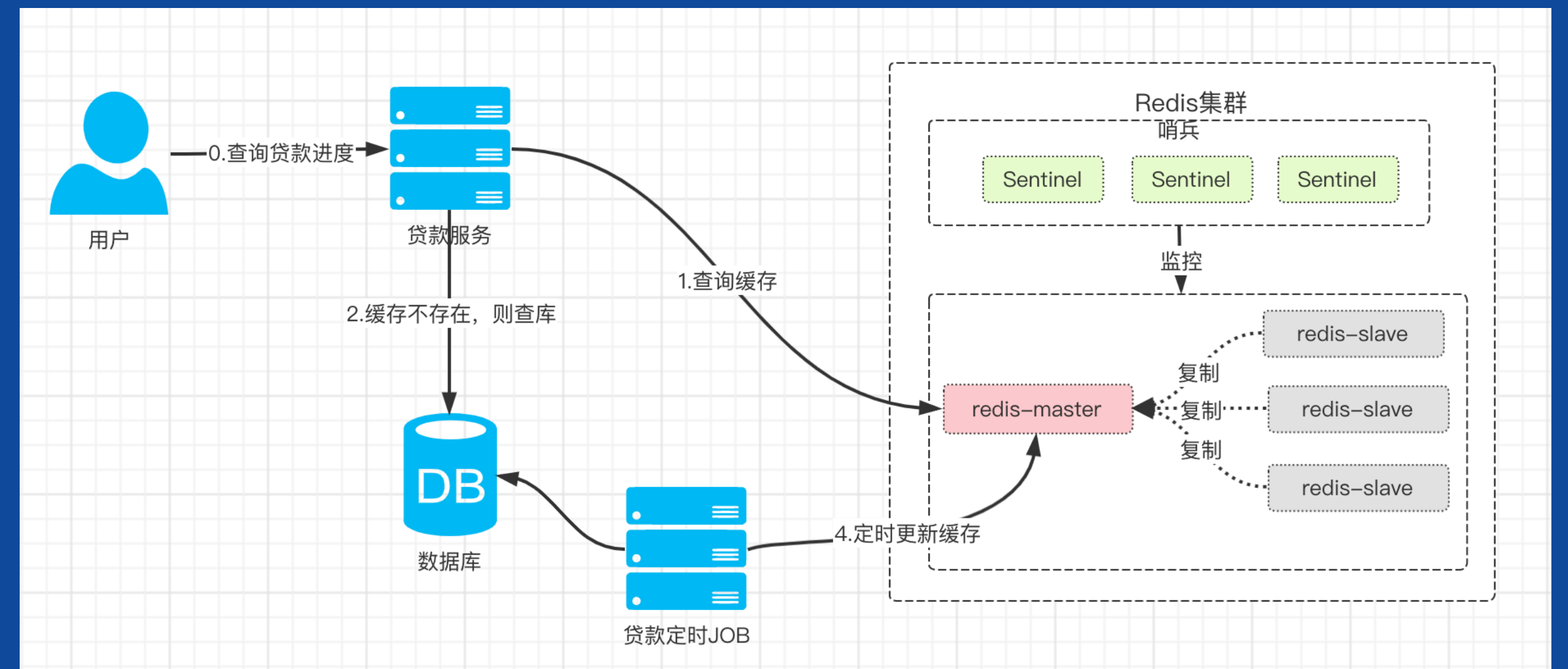
RocketMQ

- 服务之间的解耦
- 异步的处理提升系统性能
- 银行系统分布式事务



Redis缓存实践

- **高性能，高吞吐**，读的速度是110000次/s,写的速度是81000次/s；
- **丰富的数据类型**：Redis支持二进制案例的Strings, Lists, Hashes, Sets 及 Ordered Sets 数据类型操作；
- **原子性**：Redis的所有操作都是原子性的，同时Redis还支持对几个操作全并后的原子性执行；

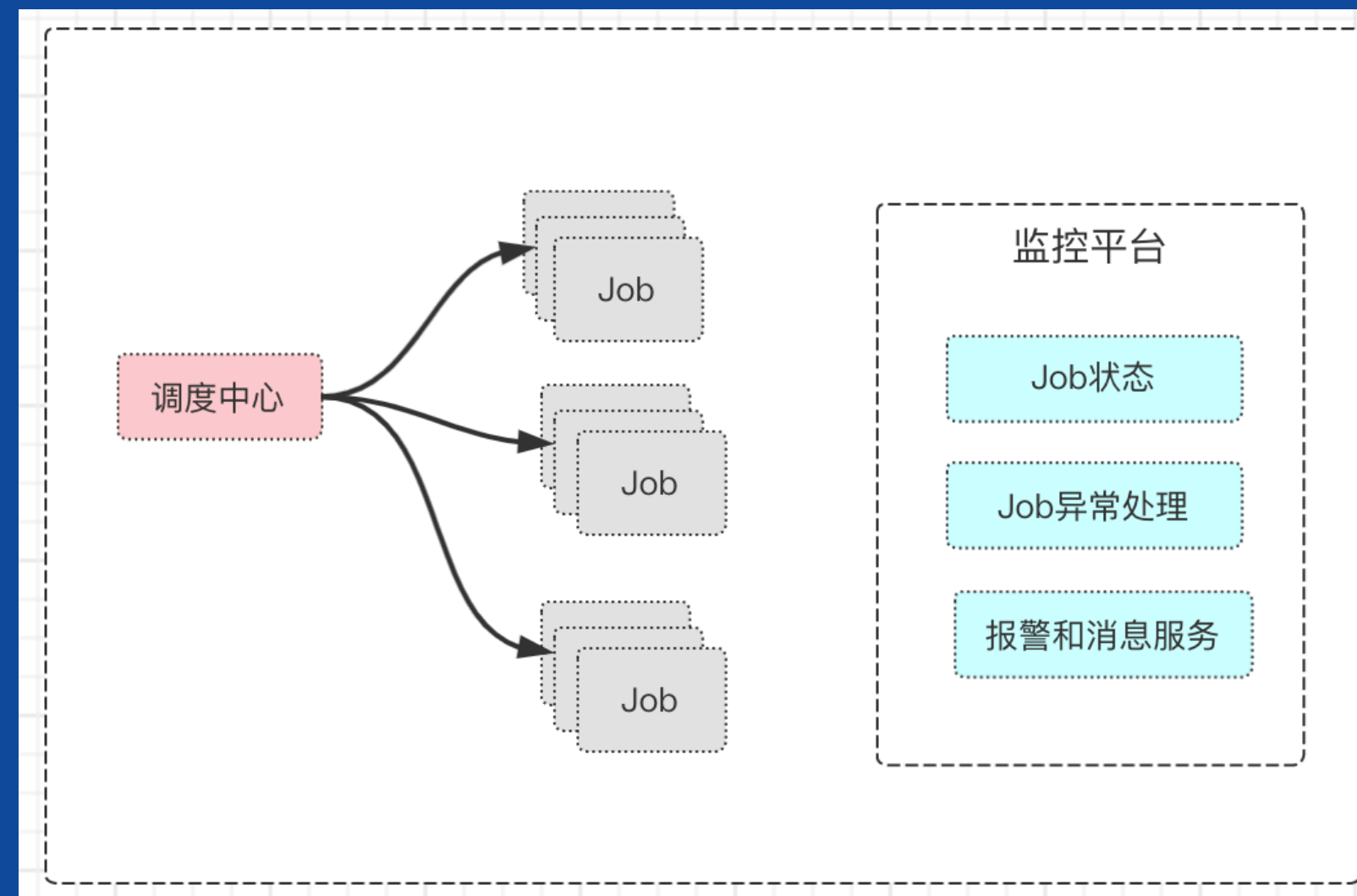


缓存实践



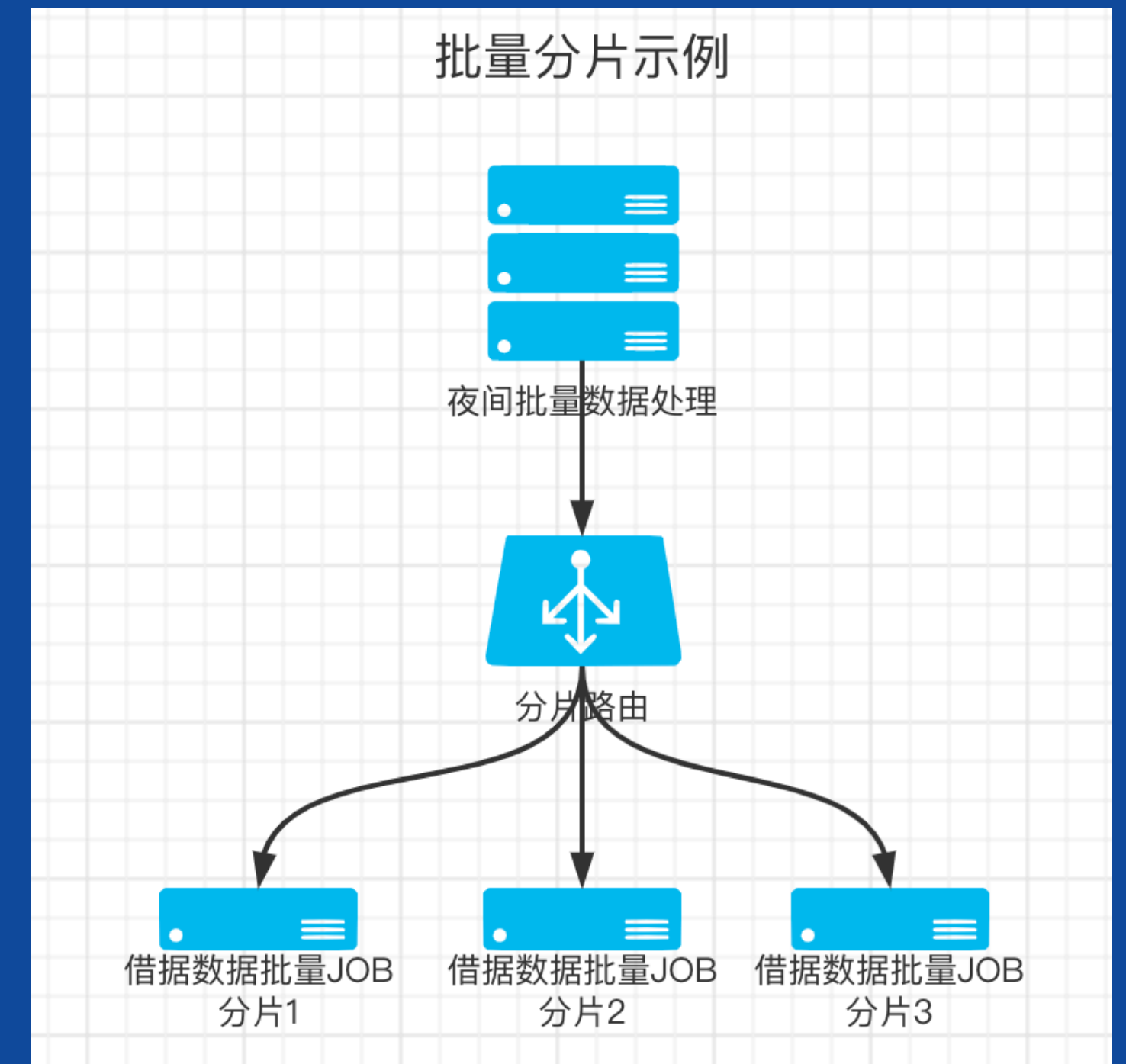
实践：JOB框架

- 数据量大
- 账务场景复杂
- 分布式调度
- 分片设计
- 监控



互联网消费金融作业Job的监控，涉及到的方面：

- 1) 作业的进度监控；
- 2) 作业状态监控，是否正常或异常；
- 3) 异常分类与报警；
- 4) 消息通知。



- 技术选型：自研 or 开源？
- Quartz、elastic-job、TBSchedule、**Saturn**、xxl-job

大纲

- 互联网消费金融架构的特性及面临的挑战
- 平安消费金融平台的服务化架构方案演进（单体-SOA-微服务）
- 平安消费金融平台微服务中间件选型及演进
- 平安消费金融平台架构演进面临的常见问题及思考

体会

- 对于传统银行、金融集团来说，**老旧系统**大量存在，系统繁多，依赖关系复杂，对旧系统的改造、监控困难；
- **安全、合规**放在第一位
- 节奏上循序渐进**小步快跑**，保持长久稳定性
- 并行验证，灰度上线
- 重要功能加上开关

架构升级带来的好处

我们下面总结一下，一个好的微服务架构设计和服务治理能为互联网消费金融业务领域带来什么好处。

- 使互联网消费金融业务系统架构上更加**清晰**，每个模块&微服务集群的职责明确，业务和系统的**边界明确**；
- 核心模块**稳定**，避免突发流量引起的雪崩
- 系统易**拓展**、易**伸缩**
- 开发**管理方便**、团队整体能力提升

THANKS

—
Global
Architect Summit





关注 InfoQ Pro 服务号

你将获得：

- ✓ InfoQ 技术大会讲师 PPT 分享
- ✓ 最新全球 IT 要闻
- ✓ 一线专家实操技术案例
- ✓ InfoQ 精选课程及活动
- ✓ 定期粉丝专属福利活动